UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

Trabajo Final Integrador

Km0 Automotores

Portal web multimarca de autos 0 km con reserva de test drive en tiempo real, escuela de manejo y gestión de solicitudes de compra

Informe Técnico Final

Integrantes

Agustín Hurtado DNI: **[COMPLETAR]**

Valentín Gonzalez Puleo DNI: **[COMPLETAR]**

Luciano Wittmund DNI: **[COMPLETAR]**

Docente tutor

Sergio Andrés Antonini

Córdoba, noviembre de 2026

Resumen ejecutivo

Las concesionarias de autos 0 km coordinan las pruebas de manejo por teléfono, mensajería o visita presencial, sin una vista en tiempo real de qué vehículos de demostración y qué vendedores están disponibles. Esto genera demoras para el comprador, turnos superpuestos y pérdida de potenciales clientes, y deja a la concesionaria sin información sobre la demanda real de cada modelo.

Como solución, el equipo desarrolló Km0 Automotores, un portal web para una concesionaria multimarca ficticia de Córdoba. El sistema permite explorar un catálogo de veintiún modelos de seis marcas con fotos, un visor tridimensional y un comparador; reservar una prueba de manejo eligiendo un horario realmente libre, con asignación automática de la unidad y del vendedor; reservar clases de manejo; y pedir un presupuesto de compra con simulación de cuotas. Los vendedores e instructores cuentan con una agenda con alertas, y la administración con un tablero de métricas y el mantenimiento completo del catálogo.

La aplicación se construyó con React y TypeScript en el navegador, Spring Boot en el servidor y PostgreSQL como base de datos, con autenticación mediante tokens firmados. La imposibilidad de reservar dos veces el mismo horario está garantizada por la propia base de datos, incluso ante pedidos simultáneos.

El sistema cumple los requerimientos funcionales planteados, cuenta con pruebas automatizadas de las reglas de negocio, de la interfaz y de la comunicación completa con la base de datos, que se ejecutan en cada cambio, y está publicado en la nube: el frontend en Vercel, el backend en Render y la base de datos en Neon.

Se espera que una solución de este tipo reduzca el tiempo de coordinación de las pruebas de manejo, elimine los turnos superpuestos y brinde a la concesionaria datos para decidir qué modelos exhibir y cómo distribuir a su personal.

Palabras clave: reserva de turnos, concesionaria, aplicación web, Spring Boot, React, PostgreSQL.

Índice general

1. Introducción **1**
2. Descripción del problema y análisis de requerimientos **2**
2.1 Descripción del problema 2
2.2 Requerimientos funcionales 4
2.3 Requerimientos no funcionales 5
2.4 Casos de uso 5
3. Marco teórico y tecnologías seleccionadas **7**
3.1 Marco teórico 7
3.2 Tecnologías seleccionadas y justificación 9
4. Descripción del proceso de desarrollo **10**
4.1 Metodología de desarrollo adoptada 10
4.2 Planificación y cronograma 11
4.3 Narrativa del proceso de desarrollo 12
4.3.1 Etapa de relevamiento y análisis 12
4.3.2 Etapa de diseño 13
4.3.3 Etapas de implementación 13
4.3.4 Etapa de pruebas y validación 14
4.3.5 Etapa de despliegue 14
4.4 Control de versiones 14
5. Arquitectura y decisiones de diseño **15**
5.1 Diagrama de arquitectura general 15
5.2 Descripción de la arquitectura 15
5.3 Decisiones de diseño relevantes 16
5.4 Modelo de datos 18
5.5 Diagramas de secuencia y de estados 19
5.6 Diseño de la interfaz de usuario 21
6. Implementación del sistema **22**
6.1 Estructura del proyecto 22
6.2 Módulos principales 22
6.3 Integración con sistemas externos 24
7. Manual de usuario **25**
7.1 Introducción al manual 25
7.2 Acceso al sistema 25
7.3 Descripción de funcionalidades 26
7.4 Preguntas frecuentes 30
8. Manual de implementación **30**
8.1 Requisitos del entorno 30
8.2 Variables de entorno y configuración 30
8.3 Procedimiento de instalación 31
8.4 Actualización del sistema 33
8.5 Respaldo y recuperación 33
8.6 Monitoreo y logs 33
9. Pruebas y validación **34**
9.1 Estrategia de pruebas 34
9.2 Casos de prueba 35
9.3 Métricas de calidad 36
9.4 Validación con usuarios 37
10. Conclusiones **38**
10.1 Resultados obtenidos 38
10.2 Lecciones aprendidas 38
10.3 Limitaciones del sistema actual 39
10.4 Trabajo futuro 39
11. Referencias bibliográficas **39**
Anexo A. Uso de inteligencia artificial **40**

Índice de figuras

Figura 1. Arquitectura general del sistema (modelo C4, nivel de contenedores)..... 15
Figura 2. Capas del backend y recorrido de un pedido..... 16
Figura 3. Diagrama entidad-relación (PostgreSQL 16)..... 18
Figura 4. Secuencia de reserva de un test drive con el caso de concurrencia..... 20
Figura 5. Secuencia de solicitud de compra..... 20
Figura 6. Estados de un turno de test drive o clase de manejo..... 21
Figura 7. Pantalla principal del sitio..... 25
Figura 8. Catálogo con filtros (izquierda) y ficha de un modelo con el visor 3D (derecha)..... 26
Figura 9. Confirmación de la reserva (izquierda) y "Mis turnos" (derecha)..... 27
Figura 10. Comparador de versiones (izquierda) y reserva de clase de manejo (derecha)..... 27
Figura 11. Formulario "Quiero comprarlo" con la simulación de cuotas (izquierda) y presupuesto listo para imprimir (derecha)..... 28
Figura 12. Agenda del vendedor con alertas..... 28
Figura 13. Tablero de métricas (izquierda) y listado de reservas con filtros (derecha)..... 29
Figura 14. Seguimiento de solicitudes de compra (izquierda) y edición de un modelo del catálogo (derecha) 29

Índice de tablas

Tabla 1. Análisis de soluciones existentes....3
Tabla 2. Partes interesadas y su necesidad principal....3
Tabla 3. Requerimientos funcionales....4
Tabla 4. Requerimientos no funcionales....5
Tabla 5. CU-01 Reservar un test drive....5
Tabla 6. CU-03 Solicitar un presupuesto de compra....6
Tabla 7. Tecnologías del backend y la base de datos....9
Tabla 8. Tecnologías del frontend, calidad y despliegue....9
Tabla 9. Roles del equipo....11
Tabla 10. Cronograma por incremento: planificado y realizado....11
Tabla 11. ADR-01 Uso de PostgreSQL como sistema de gestión de base de datos....16
Tabla 12. ADR-03 Autenticación con JWT sin sesión en el servidor....17
Tabla 13. ADR-04 Índices únicos parciales para impedir turnos superpuestos....17
Tabla 14. ADR-06 Visor 3D generado por código en lugar de modelos 3D de cada auto....17
Tabla 15. Entidades principales y sus relaciones....18
Tabla 16. Módulos del sistema....22
Tabla 17. Servicios externos....24
Tabla 18. Perfiles de usuario....25
Tabla 19. Mensajes frecuentes y cómo resolverlos....29
Tabla 20. Requisitos del entorno....30
Tabla 21. Variables de entorno....31
Tabla 22. Errores comunes en el log y su solución....34
Tabla 23. Tipos de prueba incluidos....34
Tabla 24. Casos de prueba unitarios representativos....35
Tabla 25. Casos de prueba de integración representativos (API contra PostgreSQL)....35
Tabla 26. Casos de prueba de sistema (manuales)....36
Tabla 27. Métricas de calidad....36
Tabla 28. Defectos encontrados por incremento....36
Tabla 29. Cumplimiento de los objetivos específicos....38

1. Introducción

El mercado de autos 0 km en Argentina se apoya en concesionarias que, cada vez con más frecuencia, venden varias marcas desde un mismo local. El recorrido de un comprador suele empezar en internet: compara modelos, versiones y precios en los sitios de las marcas o en portales de clasificados. Sin embargo, el paso que más influye en la decisión de compra, manejar el auto, sigue coordinándose por teléfono, por WhatsApp o en una visita presencial. Los sitios oficiales de las marcas ofrecen un formulario de "solicitar test drive" que no confirma un horario: genera un pedido de contacto que un vendedor responde después, a veces horas o días más tarde.

Esta forma de trabajo afecta a los dos lados del mostrador. El comprador pierde tiempo coordinando y muchas veces abandona el interés antes de que lo llamen. La concesionaria, por su parte, administra las agendas de vendedores y de unidades de demostración en planillas o cuadernos separados, lo que produce turnos superpuestos, unidades reservadas dos veces y ninguna información sobre qué modelos generan más interés o qué clientes no se presentan. Si además la concesionaria ofrece clases de manejo, esa agenda suele manejarse por fuera del resto de la operación.

El presente trabajo propone **Km0 Automotores**, un portal web para una concesionaria multimarca ficticia de la ciudad de Córdoba que concentra en un único sistema el catálogo de modelos, la reserva de test drive con disponibilidad real, la escuela de manejo, el seguimiento de los interesados en comprar y un panel de métricas para la administración. El cliente es de inventiva propia, inspirado en el funcionamiento de concesionarias reales de la región.

Objetivo general. Desarrollar una aplicación web que permita a los clientes de una concesionaria multimarca reservar test drives y clases de manejo con disponibilidad en tiempo real, y a la concesionaria gestionar sus agendas, su catálogo y sus potenciales ventas desde un único lugar.

Objetivos específicos. Para alcanzar el objetivo general se plantearon cinco metas verificables:

1. Publicar un catálogo navegable de al menos 15 modelos de 5 marcas, con filtros, ficha técnica por versión, galería de fotos, visor 3D y comparador de hasta tres versiones.
2. Permitir que un cliente registrado reserve un test drive eligiendo modelo, sucursal, día y horario, mostrando solo los horarios en los que hay una unidad y un vendedor libres, y garantizando que nunca se asignen dos turnos a la misma unidad o al mismo vendedor en el mismo horario.
3. Ofrecer a vendedores e instructores una agenda semanal con alertas de turnos sin confirmar y la posibilidad de registrar la asistencia del cliente.

4. Brindar a la administración un tablero con métricas de demanda (modelos y horarios más pedidos, asistencia, turnos por sucursal), el listado completo de reservas y el mantenimiento del catálogo, las sucursales, las unidades y los empleados.
5. Desplegar el sistema en la nube con integración continua y una batería de pruebas automatizadas (unitarias, de integración y de componentes) que se ejecute en cada cambio.

Alcance y limitaciones. El sistema incluye los módulos de autenticación por roles, catálogo, sucursales y unidades de demostración, reserva de test drive, panel del vendedor, panel del instructor, escuela de manejo, métricas, recordatorios automáticos y, como extensión posterior, la solicitud de compra con simulación de financiación y presupuesto imprimible. Quedan fuera del alcance el cobro en línea (pasarela de pagos), la aprobación real de créditos con entidades financieras, la integración con sistemas de gestión de stock de las automotrices, las aplicaciones móviles nativas y los modelos 3D exactos de cada vehículo. Los datos de la concesionaria, sus empleados y sus clientes son ficticios.

Organización del informe. La sección 2 describe el problema, los interesados y los requerimientos. La sección 3 presenta los conceptos que sustentan las decisiones del proyecto y justifica cada tecnología elegida. La sección 4 narra el proceso de desarrollo, con su metodología, cronograma y desvíos. La sección 5 documenta la arquitectura y las decisiones de diseño en formato de registros de decisión. La sección 6 describe la implementación por módulos. Las secciones 7 y 8 son los manuales de usuario y de implementación. La sección 9 presenta la estrategia de pruebas y sus métricas, y la sección 10 cierra con las conclusiones. Al final se incluyen las referencias y los anexos.

2. Descripción del problema y análisis de requerimientos

2.1 Descripción del problema

En una concesionaria multimarca típica, un test drive requiere coordinar tres recursos al mismo tiempo: un cliente, una unidad de demostración de la versión que quiere probar y un vendedor que lo acompañe. Hoy esa coordinación es manual. El cliente escribe o llama, el vendedor revisa una planilla o pregunta a sus compañeros qué unidad está libre, y recién entonces propone un horario. Si el cliente no puede, el intercambio vuelve a empezar. No existe una vista en tiempo real de qué unidades y vendedores están disponibles, por lo que es habitual que dos vendedores comprometan la misma unidad en el mismo horario.

El relevamiento de competidores realizado durante la propuesta del proyecto mostró que ninguna de las alternativas existentes resuelve el problema completo (Tabla 1). Los sitios de las marcas solo

generan pedidos de contacto, las concesionarias multimarca de la región ofrecen comparadores pero no reservas con horario propio, y los portales de clasificados no gestionan turnos.

Tabla 1. *Análisis de soluciones existentes*

Tipo y ejemplos	Qué ofrece	Qué no ofrece
Sitios de marca (Toyota, Volkswagen, Ford Argentina)	Formulario de "solicitar test drive" por modelo; un vendedor contacta después	Turno confirmado en el momento; disponibilidad horaria real
Concesionaria multimarca (Autocity)	Comparador de hasta 3 autos, pista de test drive, preturno de service	Reserva online de test drive con horario; escuela de manejo integrada
Portales y clasificados (MercadoLibre Autos, DeMotores)	Listado y comparación de fichas de muchas marcas	Turnos o contacto directo con una concesionaria puntual

Partes interesadas. Se identificaron cuatro grupos de usuarios con necesidades distintas, resumidos en la Tabla 2. A ellos se suma la gerencia de la concesionaria, que no opera el sistema a diario pero necesita información para decidir qué modelos exhibir, cuántas unidades de demostración mantener y cómo distribuir a los vendedores.

Tabla 2. *Partes interesadas y su necesidad principal*

Parte interesada	Rol en el sistema	Necesidad e impacto del problema actual
Cliente o comprador	Explora el catálogo, reserva test drives y clases, pide presupuestos	Quiere probar el auto sin llamar ni esperar respuesta. Hoy pierde tiempo y a veces abandona la compra.
Vendedor	Atiende los test drives y da seguimiento a los interesados en comprar	Necesita una agenda única y saber a quién llamar. Hoy sufre turnos superpuestos y pierde potenciales clientes por demora.
Instructor de manejo	Dicta las clases de la escuela	Necesita ver solo sus clases. Hoy su agenda está separada del resto.
Administrador	Mantiene catálogo, sucursales, unidades y empleados; consulta métricas	Necesita datos de demanda y ausentismo. Hoy no tiene información consolidada.

El impacto se resume en tres efectos: tiempo perdido por el comprador al coordinar, turnos duplicados o superpuestos por la falta de una agenda centralizada, y pérdida de potenciales compradores por demora en la respuesta. Como el cliente del proyecto es ficticio, no se cuenta con mediciones propias de estos efectos; el relevamiento se basó en el análisis de los sitios existentes y en la experiencia de los integrantes como compradores.

2.2 Requerimientos funcionales

La Tabla 3 lista los requerimientos funcionales con su prioridad y el criterio con el que se verifica que están cumplidos. La prioridad sigue la definida en la segunda entrega: alta para lo imprescindible para reservar un test drive, media para la gestión interna y baja para las extensiones.

Tabla 3. *Requerimientos funcionales*

ID	Descripción	Prior.	Criterio de aceptación
RF-01	Registro de clientes e inicio de sesión con email y contraseña	Alta	Un usuario nuevo se registra, inicia sesión y es dirigido a la pantalla de su rol
RF-02	Control de acceso por rol (cliente, vendedor, instructor, administrador)	Alta	Una acción no permitida para el rol devuelve 403 y la pantalla no se muestra
RF-03	Catálogo con filtros por marca, carrocería, combustible y precio	Alta	Al combinar filtros solo se listan los modelos que los cumplen
RF-04	Ficha de modelo con versiones, precios, fotos, colores y visor 3D	Alta	El auto del visor gira y cambia al color elegido
RF-05	Comparador de hasta tres versiones	Media	Se muestran las especificaciones en paralelo y se resaltan las diferencias
RF-06	Consulta de horarios libres por modelo, sucursal y día	Alta	Un horario se ofrece solo si hay al menos una unidad y un vendedor libres
RF-07	Reserva de test drive con asignación automática de unidad y vendedor	Alta	El turno queda PENDIENTE con unidad y vendedor; dos reservas del mismo recurso y horario no pueden coexistir
RF-08	Cancelación de turnos por el cliente antes de su inicio	Alta	El turno pasa a CANCELADO y el horario vuelve a estar disponible
RF-09	Agenda semanal del vendedor con alertas de turnos sin confirmar	Media	Los turnos pendientes de hoy a pasado mañana aparecen destacados
RF-10	Cambio de estado del turno (confirmar, realizado, ausente, cancelado)	Media	Solo se ofrecen las transiciones válidas; asistencia solo después del inicio
RF-11	Recordatorio automático el día anterior al turno	Media	Cada turno de mañana recibe un único aviso
RF-12	Tablero de métricas para el administrador	Media	Muestra modelos y horarios más pedidos, asistencia y turnos por sucursal en el período elegido
RF-13	Listado de todas las reservas con filtros y exportación a Excel	Media	El archivo descargado abre en Excel con las mismas filas que la pantalla
RF-14	Administración de marcas, modelos, versiones, fotos, colores, sucursales, unidades y empleados	Alta	Los cambios se reflejan en el catálogo público sin reiniciar el sistema
RF-15	Reserva de clases de manejo con instructor elegido o asignado	Baja	Un instructor no recibe dos clases en el mismo horario
RF-16	Solicitud de compra con simulación de cuotas y presupuesto imprimible	Baja	La cuota calculada coincide entre pantalla y servidor; el presupuesto se imprime sin menús
RF-17	Seguimiento de solicitudes de compra por el vendedor y el administrador	Baja	Los estados siguen el embudo NUEVA, CONTACTADA, RESERVADA, VENDIDA o

ID	Descripción	Prior.	Criterio de aceptación
			DESCARTADA

2.3 Requerimientos no funcionales

Tabla 4. *Requerimientos no funcionales*

Atributo	Requerimiento	Cómo se aborda
Rendimiento	Las pantallas principales responden en menos de 2 segundos con carga normal (decenas de usuarios simultáneos)	Consultas con join fetch y carga en lote para evitar N+1; índices en las columnas de búsqueda; three.js se descarga solo en las páginas con visor
Seguridad	Contraseñas nunca en texto plano; cada acción valida identidad y rol; un usuario no puede operar sobre datos ajenos	BCrypt, tokens firmados con expiración de 24 h, permisos por rol en cada endpoint, verificación de propiedad en los servicios, HTTPS en producción
Disponibilidad	Disponible durante el horario comercial de las sucursales (objetivo 99 % mensual)	Plataformas administradas (Vercel, Render). El plan gratuito suspende el backend sin uso: ver limitaciones en 10.3
Escalabilidad	Sumar sucursales, marcas o modelos sin cambiar código	Todo el catálogo y los horarios son datos administrables; backend sin estado, puede ejecutarse en varias instancias
Usabilidad	Utilizable en celular y computadora; accesible con teclado y lector de pantalla	Diseño adaptable, foco visible, textos alternativos, análisis estático de accesibilidad con oxlint
Mantenibilidad	Un integrante nuevo entiende y modifica un módulo sin romper otros	Arquitectura en capas, reglas de negocio aisladas y probadas, migraciones versionadas, integración continua, guía del código

2.4 Casos de uso

Se documentan los flujos de mayor valor para el negocio. Los dos principales (CU-01 y CU-03) se presentan con el formato completo de casos de uso: actor, precondiciones, flujo principal, flujos alternativos y postcondiciones.

Tabla 5. *CU-01 Reservar un test drive*

Campo	Detalle
Actor	Cliente
Precondiciones	El cliente inició sesión. El modelo tiene al menos una unidad de demostración activa en la sucursal elegida.
Flujo principal	1. El cliente elige modelo y sucursal. 2. Elige un día; el sistema muestra los horarios con lugares libres. 3. Elige un horario y opcionalmente escribe un comentario.

Campo	Detalle
	4. Confirma la reserva. 5. El sistema valida la fecha, asigna la primera unidad libre y el vendedor libre con menos turnos ese día, y guarda el turno. 6. El sistema muestra la confirmación con el vendedor asignado.
Flujos alternativos y de excepción	• 2a. La sucursal no atiende ese día (domingo, sábado sin horario): se informa el motivo y no se muestran horarios. • 4a. El cliente no inició sesión: se lo lleva a ingresar o registrarse y se conserva la reserva elegida. • 5a. Otro cliente tomó el horario un instante antes: se informa "Ese horario se acaba de ocupar" (409) y se actualiza la grilla. • 5b. El cliente ya tiene otro turno a esa hora: se rechaza con el motivo (422).
Postcondiciones	Existe un turno PENDIENTE que ocupa la unidad y al vendedor en ese horario; aparece en "Mis turnos" del cliente y en la agenda del vendedor.

CU-02 Gestionar un turno desde la agenda. El vendedor (o el administrador, para cualquier turno) abre su panel y ve arriba las alertas de turnos sin confirmar. Llama al cliente y presiona "Confirmar"; el turno pasa a CONFIRMADO. Una vez iniciado, marca "Vino" o "No vino" y el turno pasa a REALIZADO o AUSENTE. Si intenta registrar la asistencia antes del inicio, el sistema lo rechaza (422); si intenta modificar un turno de otro vendedor, recibe 403. El estado registrado alimenta las métricas de asistencia.

Tabla 6. *CU-03 Solicitar un presupuesto de compra*

Campo	Detalle
Actor	Cliente
Precondiciones	El cliente inició sesión y está en la ficha de un modelo.
Flujo principal	1. Presiona "Quiero comprarlo". 2. Elige versión, color y forma de pago (contado, financiado o plan de ahorro). 3. Si es financiado, ajusta el anticipo y la cantidad de cuotas; el sistema muestra la cuota estimada. 4. Indica si entrega un usado, elige la sucursal y envía. 5. El sistema recalcula y valida la financiación, copia el precio vigente, asigna el vendedor con menos solicitudes abiertas y muestra el presupuesto.
Flujos alternativos y de excepción	• 3a. El anticipo es menor al 30 % o el plazo no se ofrece: se rechaza con el motivo (422). • 5a. Ya tiene una solicitud abierta de esa versión: 409. • El cliente desiste mientras la solicitud está NUEVA o CONTACTADA: presiona "Ya no me interesa".
Postcondiciones	Existe una solicitud NUEVA visible para el cliente, su vendedor y el administrador; el presupuesto se puede imprimir o guardar en PDF.

La reserva de una clase de manejo sigue el mismo flujo que CU-01, con el instructor como recurso limitado en lugar de la unidad y el vendedor; el cliente puede además elegir el instructor y el tipo de caja (manual o automática).

3. Marco teórico y tecnologías seleccionadas

3.1 Marco teórico

Arquitectura cliente-servidor y en capas. Km0 Automotores sigue el modelo cliente-servidor: una aplicación de página única (SPA, *single page application*) que se ejecuta en el navegador y un servidor que concentra los datos y las reglas. Del lado del servidor se aplica la arquitectura en capas que describe Fowler (2002): la capa de presentación recibe los pedidos, la capa de dominio aplica las reglas de negocio y la capa de acceso a datos se comunica con la base. Cada capa depende solo de la inferior, lo que permite probarlas por separado y cambiar una sin afectar a las otras. En el proyecto, estas capas son los paquetes `controller`, `service` y `repository` de Spring Boot.

Servicios REST. La comunicación entre el navegador y el servidor sigue el estilo arquitectónico REST (*Representational State Transfer*) definido por Fielding (2000): los datos se exponen como recursos identificados por URL (`/api/modelos/2`), se operan con los verbos de HTTP (GET para consultar, POST para crear, PATCH para modificar) y cada pedido es independiente del anterior. Esta última propiedad, la ausencia de estado de sesión en el servidor, es la que permite que el backend se reinicie o se ejecute en varias instancias sin que los usuarios pierdan su sesión.

Autenticación sin estado con JWT. Para identificar al usuario en cada pedido sin guardar sesiones se utiliza JSON Web Token (JWT), un formato estándar de token firmado (Jones et al., 2015). Al iniciar sesión, el servidor entrega un token que contiene el identificador del usuario, su rol y su fecha de vencimiento, firmado con una clave secreta. El navegador lo envía en el encabezado `Authorization` de cada pedido y el servidor verifica la firma antes de procesarlo. Las contraseñas se almacenan con BCrypt, una función de derivación con sal y costo configurable que vuelve impracticable recuperar la contraseña a partir del valor guardado, en línea con las recomendaciones de OWASP (2021) sobre fallas criptográficas y de autenticación.

Integridad de datos y concurrencia. El problema central del proyecto es de concurrencia: dos clientes pueden intentar reservar la misma unidad en el mismo horario casi al mismo tiempo. Una validación en el código ("¿está libre?") no alcanza, porque ambos pedidos pueden consultar antes de que cualquiera guarde. Los sistemas de bases de datos relacionales resuelven este tipo de problemas con restricciones de integridad que la propia base hace cumplir dentro de transacciones ACID (atomicidad, consistencia, aislamiento y durabilidad). PostgreSQL ofrece además índices

únicos parciales, que exigen unicidad solo sobre las filas que cumplen una condición (PostgreSQL Global Development Group, s. f.). El proyecto usa esta característica para que la unicidad del horario se aplique solo a los turnos no cancelados (ver ADR-04 en la sección 5.3).

Evolución controlada del esquema. El esquema de la base cambia a lo largo del proyecto, y debe ser idéntico en la máquina de cada integrante, en las pruebas y en producción. Ambler y Sadalage (2006) proponen tratar cada cambio del esquema como una migración versionada que se aplica una sola vez y queda registrada, de forma análoga al control de versiones del código. Flyway implementa esta idea con scripts numerados (`V1`, `V2`, …) que se ejecutan en orden al iniciar la aplicación.

Pruebas automatizadas. La estrategia de pruebas sigue la pirámide propuesta por Cohn (2009): muchas pruebas unitarias rápidas sobre la lógica de negocio, menos pruebas de integración que verifican la colaboración entre componentes y pocas pruebas de punta a punta. Para que las reglas de negocio puedan probarse sin servidor ni base de datos, se aislaron en clases puras (`service/reglas`). Las pruebas de integración se ejecutan contra un PostgreSQL real levantado en un contenedor con Testcontainers, porque las garantías del sistema dependen de índices y restricciones que una base en memoria no reproduce.

Registro de decisiones de arquitectura. Las decisiones técnicas relevantes se documentaron como registros de decisión de arquitectura (ADR, *Architecture Decision Record*), formato propuesto por Nygard (2011) que deja constancia del contexto, las alternativas consideradas y las consecuencias aceptadas. Para los diagramas de arquitectura se utilizó el modelo C4 de Brown (s. f.), que describe un sistema en niveles de detalle creciente: contexto, contenedores, componentes y código.

Sistema francés de amortización. El módulo de solicitudes de compra simula la cuota de un crédito prendario con el sistema francés, el más utilizado por las financieras de las automotrices en el país. En este sistema la cuota es constante y se calcula como `cuota = M · i / (1 − (1 + i)^−n)`, donde *M* es el monto a financiar, *i* la tasa mensual (tasa nominal anual dividida por 12) y *n* la cantidad de cuotas. La misma fórmula está implementada en el servidor y en el navegador, y las pruebas de ambos lados verifican que den idéntico resultado.

Accesibilidad. La interfaz toma como referencia las Pautas de Accesibilidad para el Contenido Web (WCAG) 2.2 del W3C (2023): contraste suficiente, navegación completa con teclado con foco visible, textos alternativos en imágenes y celdas de tabla con texto para lectores de pantalla.

3.2 Tecnologías seleccionadas y justificación

La Tabla 7 y la Tabla 8 detallan cada tecnología con su versión, su propósito en el proyecto, la justificación frente a las alternativas evaluadas y su licencia. Todas las licencias son libres y permiten el uso académico y comercial sin costo; ninguna obliga a publicar el código del proyecto. Las versiones y capacidades de cada herramienta se tomaron de su documentación oficial (Spring, s. f.; React, s. f.; Redgate, s. f.; Testcontainers, s. f.; three.js, s. f.).

Tabla 7. *Tecnologías del backend y la base de datos*

Tecnología y versión	Propósito	Justificación y alternativas descartadas	Licencia
Java 17 (LTS)	Lenguaje del backend	Versión LTS exigida por Spring Boot 3 y conocida por el equipo. Kotlin: curva de aprendizaje; Node.js: se buscaba tipado fuerte y el ecosistema de Spring de la carrera.	GPL v2 con excepción de enlace (OpenJDK)
Spring Boot 3.5.6	Framework de la API REST	Seguridad, validación y acceso a datos integrados; es el framework que se cursa. Express o Django obligaban a armar esas piezas a mano.	Apache 2.0
Spring Security + jjwt 0.12.6	Autenticación y permisos por rol	Integración nativa con Spring. Sesión con cookie: complica el dominio cruzado; Auth0 o Firebase: dependencia externa (ADR-03).	Apache 2.0
Spring Data JPA (Hibernate 6)	Acceso a datos y mapeo de entidades	Repositorios declarativos y JPQL. JDBC directo: mucho código repetitivo.	Apache 2.0 / LGPL 2.1
PostgreSQL 16	Base de datos	Índices únicos parciales y CHECK. MySQL no tiene índices parciales; MongoDB no garantiza la unicidad del horario con la misma facilidad (ADR-01).	PostgreSQL License (tipo BSD)
Flyway 11	Migraciones del esquema	Scripts versionados aplicados al arrancar. `ddl-auto=update` no crea índices parciales ni CHECK (ADR-05).	Apache 2.0 (Community)
springdoc-openapi 2.8.9	Documentación interactiva de la API (Swagger UI)	Documentación generada desde el código, sin un archivo aparte que se desactualice.	Apache 2.0
JUnit 5, Mockito, Testcontainers 1.21.4, JaCoCo 0.8.15	Pruebas y cobertura	Estándar de Spring. Testcontainers en lugar de H2 porque H2 no reproduce los índices parciales.	EPL 2.0 / MIT
Docker	Base de datos local e imagen del backend	Levanta PostgreSQL igual en cualquier sistema; Render construye la misma imagen.	Apache 2.0

Tabla 8. *Tecnologías del frontend, calidad y despliegue*

Tecnología y versión	Propósito	Justificación y alternativas descartadas	Licencia
React 19	Interfaz de usuario	La que mejor domina el equipo. Angular: tamaño y curva; Vue: menor experiencia.	MIT
TypeScript 7	Tipado estático del	Los tipos reflejan los DTO del backend y	Apache 2.0

Tecnología y versión	Propósito	Justificación y alternativas descartadas	Licencia
	frontend	detectan errores de integración al compilar.	
Vite 8	Servidor de desarrollo y empaquetado	Arranque instantáneo y código dividido por página. Create React App está discontinuado; Next.js agrega renderizado en servidor innecesario.	MIT
React Router 8	Navegación	Rutas por pantalla y filtros guardados en la URL.	MIT
three.js 0.186	Visor 3D del auto	Liviano y flexible. Babylon.js es más pesado; un visor de Sketchfab no permite cambiar colores (ADR-06).	MIT
CSS Modules	Estilos	Estilos aislados sin dependencias. Tailwind: se priorizó un sistema de diseño propio.	MIT
Vitest 5 + Testing Library	Pruebas del frontend	Comparte la configuración de Vite. Jest requiere configuración extra para TypeScript.	MIT
oxlint 1.87	Análisis estático	Reglas de React y accesibilidad. ESLint con typescript-eslint no soporta TypeScript 7.	MIT
GitHub y GitHub Actions	Repositorio e integración continua	Obligatorio por la cátedra; pruebas en cada push sin servidores propios.	Servicio gratuito
Vercel y Render	Hosting del frontend y del backend con PostgreSQL	Planes gratuitos, despliegue automático y HTTPS. Railway y Neon son equivalentes.	Servicio gratuito

4. Descripción del proceso de desarrollo

4.1 Metodología de desarrollo adoptada

El equipo adoptó un enfoque **incremental con tablero Kanban** (Anderson, 2010). Las tareas se organizaron en un tablero de GitHub Projects con tres columnas (Pendiente, En curso, Terminado) y la regla de que cada integrante tuviera como máximo dos tareas en curso a la vez **[VERIFICAR: que el tablero exista y refleje esto]**. Cada incremento terminaba con una versión funcionando de punta a punta (base de datos, API y pantalla) que se mostraba al tutor.

Se eligió Kanban en lugar de Scrum por tres razones. Primero, el equipo se formó a comienzos de septiembre con un plazo fijo y corto (diez semanas hasta la entrega final), sin margen para ceremonias de planificación y revisión de sprints de dos semanas. Segundo, los tres integrantes combinan el proyecto con otras materias y ocupaciones, por lo que la disponibilidad variaba de semana a semana; un flujo continuo con límites de trabajo en curso se adapta mejor que un compromiso fijo por sprint. Tercero, la cátedra exigía reportar avances al tutor cada 48 a 72 horas, lo que ya funcionaba como un ciclo corto de revisión.

Adaptaciones. En lugar de reuniones diarias, el equipo se reunió por videollamada una o dos veces por semana: en cada reunión se revisaba lo terminado, se repartían las tareas siguientes según el área de cada integrante y se preparaba el avance para el tutor. Las entregas de la cátedra (propuesta, arquitectura y módulos, entrega final) se usaron como hitos que cerraban cada incremento. Además, se incorporó un asistente de inteligencia artificial como parte del flujo de desarrollo, con la regla de que todo lo generado se revisa y se prueba antes de integrarse (ver Anexo A).

Herramientas. GitHub (repositorio, tablero de GitHub Projects y GitHub Actions), mensajería del campus virtual para el contacto con el tutor, Visual Studio Code como editor, Docker Desktop para la base de datos local y Claude (Anthropic) como asistente de desarrollo.

Roles. El trabajo se dividió por capa del sistema, como resume la Tabla 9. Cada integrante fue responsable de su área de punta a punta, y todos participaron en la revisión del código y en las pruebas de los recorridos completos.

Tabla 9. *Roles del equipo*

Integrante	Rol principal	Responsabilidades
Agustín Hurtado	Frontend	Interfaz en React y TypeScript: catálogo, visor 3D, flujo de reserva, paneles y administración; diseño visual y adaptación a celular; cliente de la API; manual de usuario
Valentín Gonzalez Puleo	Backend	API REST en Spring Boot: seguridad con JWT y roles, reglas de disponibilidad y estados de turnos, solicitudes de compra, recordatorios; documentación de la API con Swagger
Luciano Wittmund	Base de datos y calidad	Modelo de datos, migraciones con Flyway y datos de ejemplo; pruebas automatizadas e integración continua; manual de implementación y documentación técnica

4.2 Planificación y cronograma

La Tabla 10 compara lo planificado en la propuesta con lo efectivamente realizado en cada incremento. Las fechas de inicio y cierre de cada incremento surgen del historial del repositorio y de los reportes al tutor.

Tabla 10. *Cronograma por incremento: planificado y realizado*

Incremento	Fechas	Planificado	Realizado y desvíos
0. Conformación y propuesta	04/09 – 18/09	Formar el equipo, definir problema, actores, alcance y stack	Cumplido. Propuesta y repositorio presentados antes del 18/09.
1. Arquitectura y	19/09 – 27/09	Diagrama entidad-relación,	Cumplido el 27/09 (segunda

Incremento	Fechas	Planificado	Realizado y desvíos
módulos		módulos con prioridad, arquitectura	entrega). El tutor pidió un DER explícito, índices en la base, documento de arquitectura y prioridad de módulos; se incorporaron en el incremento 2.
2. Base del sistema	[COMPLETAR: 28/09 – …]	Esquema con Flyway, seguridad JWT, catálogo, reserva de test drive	Cumplido. Desvío: al implementar la cancelación se rediseñó la restricción de unicidad del esquema aprobado (índices parciales, ADR-04).
3. Gestión interna y valor agregado	[COMPLETAR]	Paneles de vendedor e instructor, administración, métricas, recordatorios	Cumplido. La escuela de manejo, planificada como opcional, se adelantó porque reutilizaba la lógica de horarios.
4. Calidad y ajustes	[COMPLETAR: … – 30/10]	Pruebas, integración continua, documentación	Cumplido con agregados no planificados: fotos reales con licencia libre, listado de reservas para el admin, módulo de solicitudes de compra y correcciones surgidas de probar el sistema en Windows.
5. Despliegue y entrega final	31/10 – 14/11	Despliegue, informe técnico, video	Sistema publicado el 06/11/2026; entrega el 14/11/2026

El desvío más significativo fue de alcance y no de tiempo: el módulo de solicitudes de compra no estaba en la propuesta (la pasarela de pago y la financiación figuraban explícitamente fuera de alcance). Se incorporó como extensión acotada, sin cobro en línea, porque cerraba el recorrido natural del cliente (ver, probar, comprar) y reutilizaba la infraestructura existente de usuarios, sucursales y vendedores.

4.3 Narrativa del proceso de desarrollo

4.3.1 Etapa de relevamiento y análisis

El relevamiento se hizo sobre sistemas existentes, ya que el cliente es ficticio. Se recorrieron los sitios oficiales de Toyota, Volkswagen, Ford y Mercedes-Benz Argentina, el sitio de una concesionaria multimarca de Córdoba y portales de clasificados, registrando qué ofrece cada uno en el recorrido de compra (Tabla 1). El hallazgo principal fue que ningún sitio confirma un turno de test drive en el momento: todos generan un pedido de contacto. A partir de ahí se definió el requerimiento central (RF-06 y RF-07) y la propuesta de valor: disponibilidad real y confirmación inmediata. El segundo hallazgo fue que ninguno integra una escuela de manejo, lo que dio lugar al módulo 7.

4.3.2 Etapa de diseño

Con los requerimientos definidos, el equipo diseñó el modelo de datos en torno a la idea de que un test drive se reserva sobre una **unidad física** (`vehiculo_demo`) y no sobre una versión del catálogo: así se puede saber qué auto concreto se usa y tener varias unidades del mismo modelo. Clientes y empleados se unificaron en una sola tabla `usuario` diferenciada por rol, para tener un único inicio de sesión. La segunda entrega presentó el diagrama entidad-relación de 8 entidades, los 7 módulos con su prioridad y la arquitectura en capas. El tutor aprobó la propuesta y pidió explicitar el DER, documentar los índices y la arquitectura y ordenar los módulos por prioridad; esos documentos se completaron en `docs/` al comienzo de la implementación. No se elaboraron wireframes formales: la interfaz se diseñó directamente sobre prototipos funcionales, a partir de un sistema de tokens de diseño (sección 5.6).

4.3.3 Etapas de implementación

Incremento 2: base del sistema. El objetivo fue tener el recorrido principal funcionando: ver el catálogo y reservar un test drive. Se implementaron las migraciones V1 (esquema) y V2 (datos iniciales: 5 marcas, 15 modelos, 34 versiones, 3 sucursales, 25 unidades de demostración y 40 usuarios), la seguridad con JWT, el catálogo y el cálculo de disponibilidad. Al implementar la cancelación de turnos apareció el principal obstáculo técnico del proyecto: la restricción `UNIQUE (vehiculo_demo_id, fecha, hora_inicio)` del esquema aprobado hacía que un turno cancelado bloqueara el horario para siempre. Además, el análisis de concurrencia mostró que dos pedidos simultáneos podían pasar la validación del servicio. Ambos problemas se resolvieron con índices únicos parciales (ADR-04), cambio que se documentó para el tutor en `docs/cambios-esquema.md`. También se detectó que la validación de fechas usaba la hora del servidor (UTC) y rechazaba turnos válidos de la noche argentina; se corrigió inyectando un reloj con la zona `America/Argentina/Cordoba`.

Incremento 3: gestión interna y valor agregado. Se construyeron la agenda semanal del vendedor con alertas, el cambio de estados, el panel de administración con sus ABM, el tablero de métricas y el recordatorio automático. La escuela de manejo, prevista como opcional, se adelantó porque reutilizaba el cálculo de horarios con el instructor como recurso limitado. Para que el tablero tuviera datos desde el primer arranque se agregó la migración V3, que genera cinco semanas de turnos de ejemplo respetando las mismas reglas que la aplicación; una verificación sobre esos datos encontró un cliente con un test drive y una clase a la misma hora, y el script se corrigió para chequear ambas tablas.

Incremento 4: calidad y ajustes. Se escribieron las pruebas unitarias, de integración y de componentes, y se configuró la integración continua. Al usar el sistema en las computadoras del equipo (Windows) surgieron ajustes que no se habían visto en el entorno de desarrollo: Docker Desktop debía estar abierto antes de levantar la base, el formulario de registro desbordaba su contenedor, el menú de administración desaparecía al entrar en las agendas y el presupuesto impreso salía pegado a los bordes de la hoja. Cada caso se corrigió y se volvió a probar. En esta etapa se sumaron tres funcionalidades no planificadas: fotos reales de los modelos con licencia libre, descargadas por un script que verifica la licencia de cada imagen (salvo una foto de prensa de la S10 actual, citada como tal); un listado de todas las reservas para el administrador con exportación a Excel; y el módulo de solicitudes de compra (migración V4).

[COMPLETAR: agregar cualquier obstáculo o cambio que haya vivido el equipo y no figure acá, con su fecha y cómo se resolvió]

4.3.4 Etapa de pruebas y validación

Las pruebas acompañaron a la implementación: las reglas de horarios y de estados se probaron a medida que se escribían, y al cerrar cada incremento se recorrió el sistema completo con cada rol. Las pruebas de integración contra PostgreSQL y las de componentes del frontend se consolidaron en el incremento 4, junto con la integración continua que las ejecuta en cada push. El detalle está en la sección 9.

4.3.5 Etapa de despliegue

El sistema quedó publicado el 6 de noviembre de 2026: frontend en Vercel (https://km0-automotores-prueba.vercel.app), backend en Render (https://km0-backend-prueba.onrender.com) y PostgreSQL 16 en Neon, con publicación automática en cada cambio de la rama principal. La base está en la misma región que el backend (AWS, Ohio), y se eligió Neon porque la base gratuita de Render se elimina a los 30 días. El único problema fue de CORS: el dominio asignado por Vercel no estaba en `CORS_ORIGINS` y el navegador bloqueaba los pedidos; se resolvió agregándolo en Render. El procedimiento completo está en la sección 8.

4.4 Control de versiones

El código se versiona en GitHub (repositorio `AgusS05/km0-automotores`). Se adoptó una variante simplificada de GitFlow: la rama `main` contiene siempre una versión que funciona y es la que se despliega, y `develop` integra el trabajo en curso; cada funcionalidad se desarrolla en una rama propia (`feature/...`, `fix/...`) que se integra mediante pull request **[VERIFICAR: que esto coincida con cómo trabajaron; si todo se subió directo a main, decirlo así].** GitHub Actions ejecuta

las pruebas de backend y frontend en cada push a `main` o `develop` y en cada pull request, y marca el commit con una cruz roja si algo falla.

Los mensajes de commit se escriben en español con un prefijo **[VERIFICAR]** que indica el tipo de cambio: `feat:` para funcionalidades, `fix:` para correcciones, `docs:` para documentación y `test:` para pruebas (por ejemplo, `feat: listado de reservas para el admin`). **[COMPLETAR: insertar una captura del historial de commits (GitHub → Insights → Contributors) que muestre el trabajo de los tres integrantes]**

5. Arquitectura y decisiones de diseño

5.1 Diagrama de arquitectura general

La Figura 1 muestra el sistema en el nivel de contenedores del modelo C4. Los rectángulos dentro del límite "Km0 Automotores" son los tres contenedores del sistema; fuera del límite están los actores y los sistemas externos. Las flechas continuas son comunicaciones obligatorias y las punteadas, integraciones opcionales.

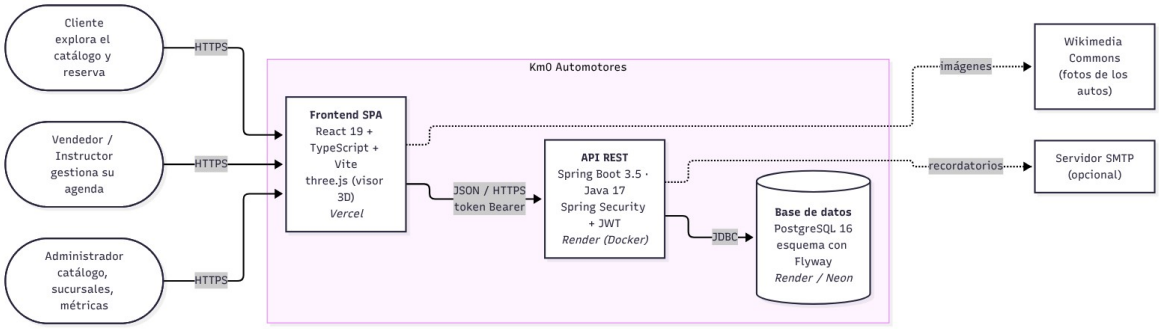


Figura 1. *Arquitectura general del sistema (modelo C4, nivel de contenedores)*
Fuente: Elaboración propia.

Los usuarios interactúan solo con el frontend, una aplicación de página única que se descarga una vez y luego se comunica con la API por HTTPS intercambiando JSON. El frontend nunca accede a la base: todo pasa por la API, que valida el token, aplica las reglas de negocio y persiste en PostgreSQL. Los dos sistemas externos son opcionales: sin servidor SMTP los recordatorios se registran en el log, y si una foto de Wikimedia Commons no carga se muestra una silueta del auto.

5.2 Descripción de la arquitectura

El patrón principal es **cliente-servidor con un backend monolítico en capas**. La Figura 2 muestra el recorrido de un pedido dentro del backend: el filtro de seguridad valida el token; el controlador valida el JSON recibido y el rol del usuario; el servicio aplica las reglas de negocio dentro de una transacción; y el repositorio ejecuta las consultas. Las reglas que no dependen de la base (cálculo de

horarios, transiciones de estado, financiación) están en clases puras del paquete `service/reglas`, sin dependencias de Spring, para poder probarlas con pruebas unitarias simples. Cualquier excepción termina en un único manejador global que devuelve siempre el mismo formato de error.

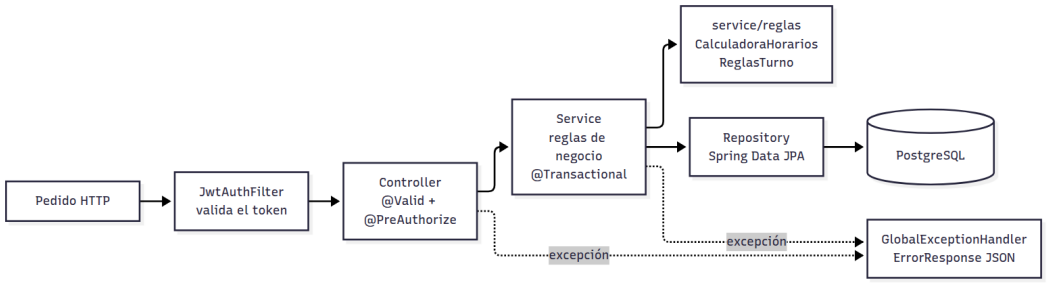


Figura 2. *Capas del backend y recorrido de un pedido*

Fuente: Elaboración propia.

Se eligió un monolito y no microservicios porque el dominio es acotado, el equipo es de tres personas y una reserva (validar, elegir unidad y vendedor, guardar) debe ser una única transacción; distribuirla entre servicios habría agregado complejidad sin beneficio (ADR-02). Frontend y backend, en cambio, sí están separados: se despliegan en plataformas distintas, se desarrollan en paralelo y se comunican solo a través del contrato de la API, documentado con Swagger.

5.3 Decisiones de diseño relevantes

Se registraron seis decisiones de arquitectura, todas documentadas en `docs/decisiones-arquitectura.md`. Dos de ellas ya se justificaron en este informe: **ADR-02**, el monolito en capas en lugar de microservicios (sección 5.2), y **ADR-05**, las migraciones versionadas con Flyway en lugar de `ddl-auto=update` de Hibernate, que no crea índices parciales ni restricciones CHECK y puede alterar la base de producción (sección 3.1). Las tablas siguientes presentan las otras cuatro en el formato ADR solicitado por la cátedra.

Tabla 11. *ADR-01 Uso de PostgreSQL como sistema de gestión de base de datos*

Campo del ADR	Contenido
Contexto	El sistema guarda un catálogo con relaciones fijas (marca, modelo, versión, unidad) y turnos que no pueden superponerse aunque dos personas reserven al mismo tiempo.
Opciones consideradas	• MongoDB: esquema flexible, pero las relaciones se resuelven en la aplicación y la unicidad entre documentos es más limitada. • MySQL: relacional y transaccional, sin índices únicos parciales. • PostgreSQL: relacional, ACID, índices únicos parciales y restricciones CHECK.
Decisión adoptada	PostgreSQL 16, por sus índices parciales y sus restricciones, y porque tiene planes gratuitos en Render, Neon y Supabase.
Consecuencias	(+) La regla "una unidad no tiene dos turnos activos a la misma hora" la garantiza la base, no solo el código.

Campo del ADR	Contenido
	(−) Requiere un servidor de base también en desarrollo, resuelto con Docker Compose.

Tabla 12. *ADR-03 Autenticación con JWT sin sesión en el servidor*

Campo del ADR	Contenido
Contexto	Hay cuatro roles y el frontend (Vercel) está en otro dominio que el backend (Render).
Opciones consideradas	• Sesión con cookie: entre dominios distintos exige configurar SameSite=None, CSRF y CORS con credenciales. • JWT en el encabezado Authorization: el servidor no guarda sesión. • Proveedor externo (Auth0, Firebase): agrega una dependencia y una cuenta de terceros.
Decisión adoptada	JWT firmado con HMAC, válido 24 horas, contraseñas con BCrypt y permisos por rol con Spring Security.
Consecuencias	(+) El backend puede reiniciarse o escalar sin perder sesiones; en cada pedido se relee el usuario, así un empleado dado de baja pierde el acceso. (−) El token se guarda en localStorage, accesible por JavaScript ante un ataque XSS; se mitiga porque React escapa el contenido. Queda como mejora pasar a cookie HttpOnly.

Tabla 13. *ADR-04 Índices únicos parciales para impedir turnos superpuestos*

Campo del ADR	Contenido
Contexto	El esquema aprobado tenía UNIQUE (vehiculo_demo_id, fecha, hora_inicio): un turno CANCELADO seguía ocupando el horario para siempre. Además, dos pedidos simultáneos podían pasar la validación del servicio antes de que cualquiera guardara.
Opciones consideradas	• Validar solo en el servicio: no resuelve la carrera. • Bloqueo pesimista (SELECT … FOR UPDATE): funciona, pero bloquea filas y complica el código. • Borrar el turno al cancelarlo: se pierde el historial de las métricas. • Índice único parcial WHERE estado <> 'CANCELADO'.
Decisión adoptada	Tres índices únicos parciales (por unidad, por vendedor y por instructor). El servicio valida primero para dar un mensaje claro y guarda con saveAndFlush; si otro pedido ganó el horario, la base rechaza el INSERT y la API responde 409.
Consecuencias	(+) La garantía vale aunque haya varias instancias del backend o un INSERT manual; cancelar libera el horario y conserva el historial. (−) Es una característica de PostgreSQL: migrar de motor requeriría otra solución.

Tabla 14. *ADR-06 Visor 3D generado por código en lugar de modelos 3D de cada auto*

Campo del ADR	Contenido
Contexto	La propuesta incluía ver los autos y girarlos. Los modelos 3D oficiales no son de uso libre y los de tiendas tienen licencias pagas o pesan decenas de megabytes.
Opciones consideradas	• Modelos glTF por auto: realistas, con problemas de licencia y de peso. • Fotos 360°: no existen fotos libres de los 15 modelos. • Auto estilizado generado con three.js según la carrocería, pintado con los colores reales.

Campo del ADR	Contenido
Decisión adoptada	Geometría generada por código con proporciones por carrocería (sedán, hatchback, SUV, pickup); las fotos reales con licencia libre se muestran en una pestaña aparte.
Consecuencias	(+) Sin problemas de licencia; agregar un modelo no requiere archivos 3D; three.js se descarga solo donde se usa. (−) No es una réplica exacta de cada modelo, y se aclara en la interfaz.

5.4 Modelo de datos

La Figura 3 presenta el diagrama entidad-relación con las claves y los atributos que intervienen en las relaciones; el detalle completo de columnas está en `docs/esquema_bd_y_modulos.md`. El modelo tiene 11 tablas: las 8 aprobadas en la segunda entrega y 3 agregadas durante la implementación (`imagen_modelo`, `color_modelo` y `solicitud_compra`).

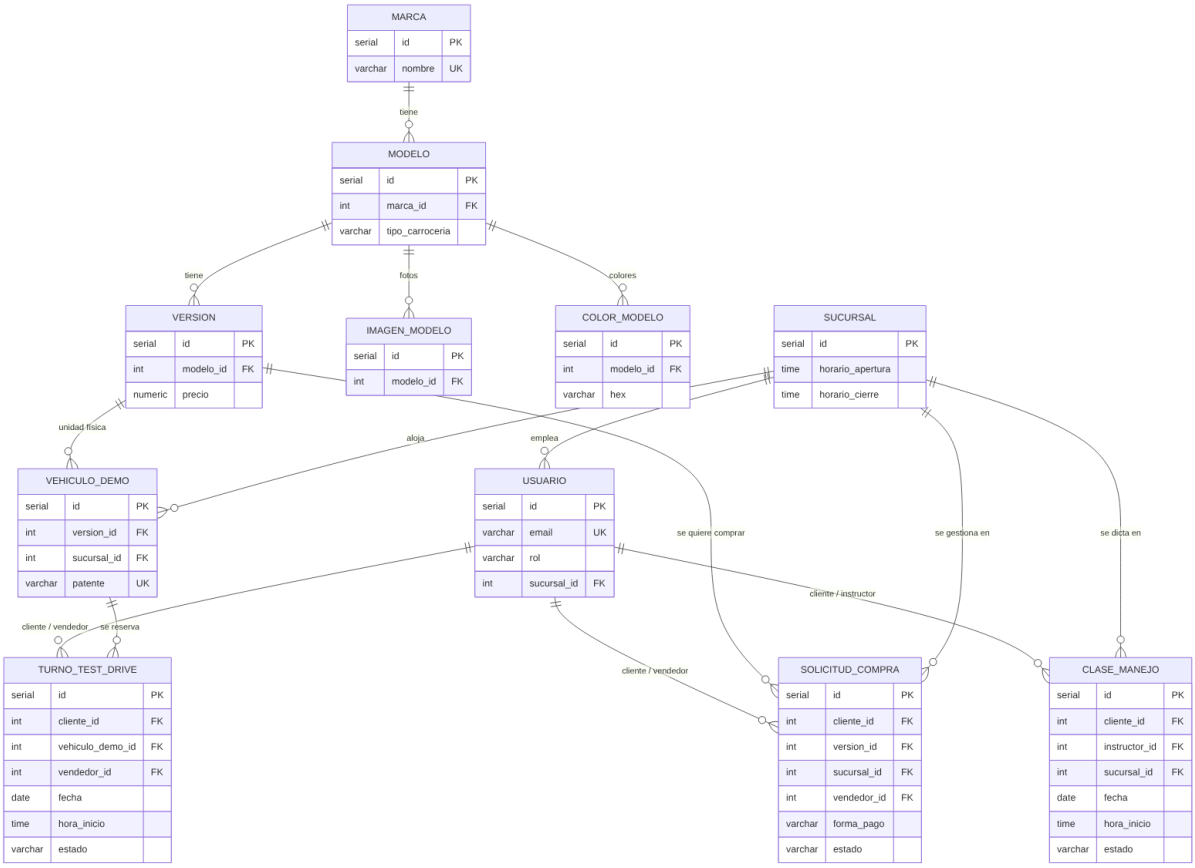


Figura 3. *Diagrama entidad-relación (PostgreSQL 16)*

Fuente: Elaboración propia.

Tabla 15. *Entidades principales y sus relaciones*

Entidad	Atributos relevantes	Relaciones
marca / modelo / version	Nombre; tipo de carrocería (define la forma en el visor 3D); precio y ficha técnica por versión	Una marca tiene muchos modelos; un modelo, muchas versiones, fotos y colores

Entidad	Atributos relevantes	Relaciones
sucursal	Dirección, teléfono, horario de apertura y cierre, cierre del sábado (nulo si no abre)	Aloja unidades de demostración y emplea vendedores e instructores
usuario	Email único, contraseña con BCrypt, rol, activo	Un cliente reserva turnos, clases y solicitudes; un empleado pertenece a una sucursal (CHECK)
vehiculo_demo	Patente única, disponible	Es una unidad física de una versión, ubicada en una sucursal
turno_test_drive	Fecha, hora de inicio y fin, estado, recordatorio enviado	Relaciona cliente, unidad y vendedor. Índices únicos parciales por unidad y por vendedor
clase_manejo	Fecha, horario, tipo de caja, estado	Relaciona cliente, instructor y sucursal. Índice único parcial por instructor
solicitud_compra	Forma de pago, copia del precio, anticipo, cuotas, cuota estimada, usado, estado	Relaciona cliente, versión, sucursal y vendedor. Un índice parcial impide dos solicitudes abiertas de la misma versión

Las reglas de integridad se aplican también en la base: restricciones CHECK sobre los valores válidos de los campos tipo lista, claves foráneas sin borrado en cascada e índices sobre las claves foráneas y las consultas frecuentes, como `(vendedor_id, fecha)` para la agenda.

5.5 Diagramas de secuencia y de estados

La Figura 4 describe el proceso más crítico del sistema: la reserva de un test drive, incluido el caso en que dos clientes eligen el mismo horario. La validación del servicio cubre el caso normal con mensajes claros; el caso de dos pedidos simultáneos lo resuelve la base de datos con el índice único parcial: el segundo INSERT falla y el cliente recibe un 409 con la grilla actualizada, en lugar de un turno duplicado.

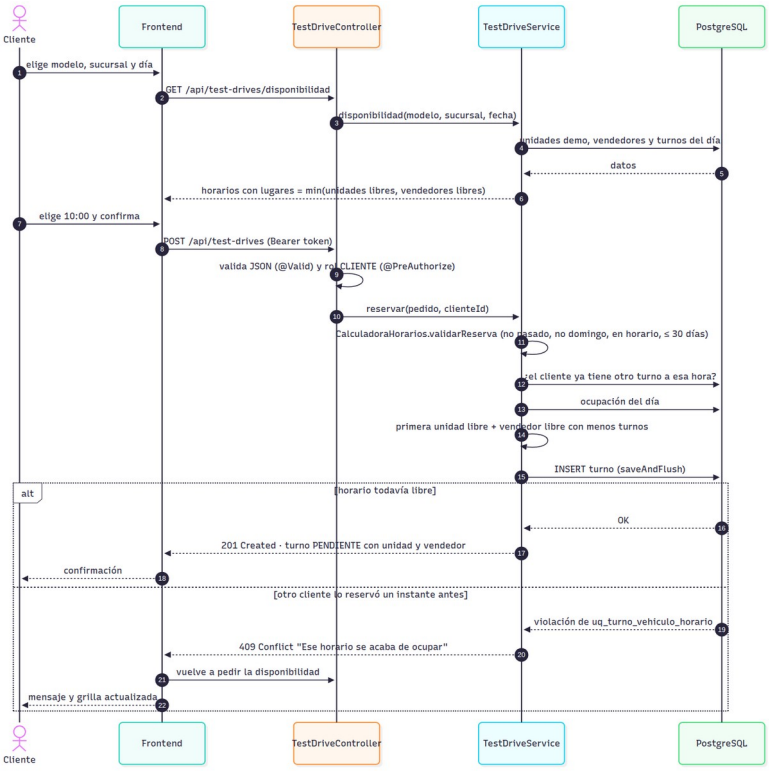


Figura 4. *Secuencia de reserva de un test drive con el caso de concurrencia*
Fuente: Elaboración propia.

La Figura 5 presenta la solicitud de compra. La cuota se simula en el navegador mientras el cliente ajusta anticipo y plazo, pero el servidor la recalcula al recibir la solicitud y no confía en el valor enviado. El precio se copia en la solicitud para que el presupuesto no cambie si después se actualiza la lista de precios.

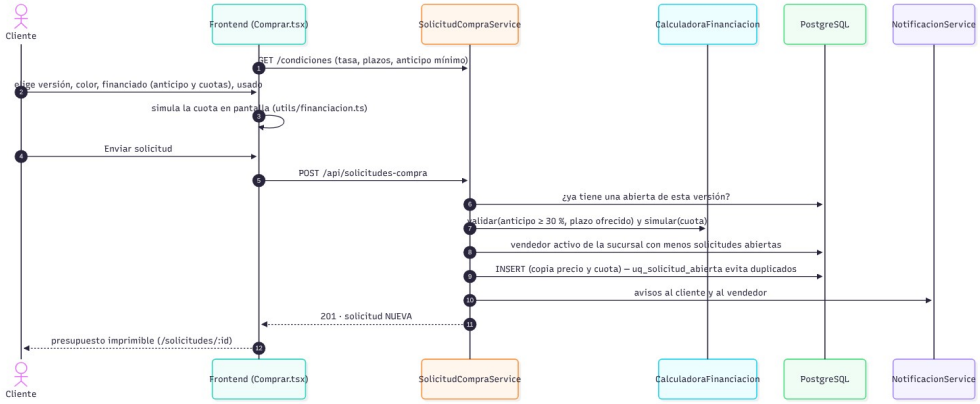


Figura 5. *Secuencia de solicitud de compra*
Fuente: Elaboración propia.

Por último, la Figura 6 muestra el ciclo de vida de un turno. Las transiciones válidas están definidas en un único lugar del backend (`EstadoTurno.java`) y el frontend las replica solo para mostrar los botones permitidos; si se intentara una transición inválida, el servidor la rechaza igualmente.

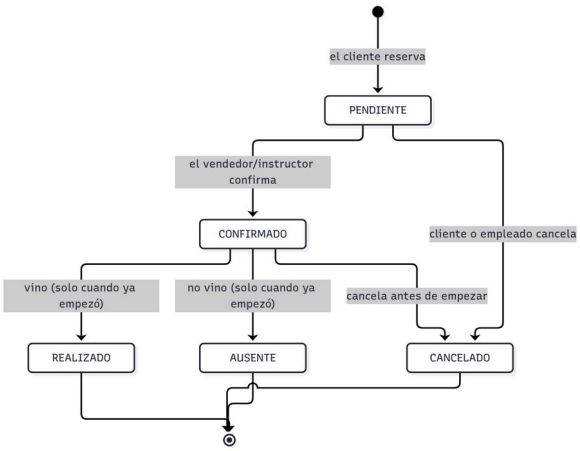


Figura 6. *Estados de un turno de test drive o clase de manejo*
Fuente: Elaboración propia.

5.6 Diseño de la interfaz de usuario

No se elaboraron wireframes formales: la interfaz se construyó directamente sobre prototipos funcionales, que se revisaban en el navegador al cerrar cada incremento. Para mantener coherencia se definió primero un sistema de tokens de diseño (`styles/tokens.css`) con colores, tipografías, espaciados y radios, que todas las pantallas reutilizan. Las decisiones principales fueron:

- **Identidad de concesionaria, no de plantilla.** Grises de salón de exhibición como base, un único color de acción (azul de patente Mercosur, #1F3A93) y un logo en forma de odómetro en cero kilómetros, con el amarillo de las líneas de ruta reservado solo para ese logo.
- **Tipografía condensada.** Barlow Condensed en títulos y Barlow en textos, una familia inspirada en la señalética vial, legible en tamaños chicos de tablas y fichas técnicas.
- **La reserva en una sola pantalla.** Los pasos (modelo, sucursal, día, horario) están numerados y visibles a la vez, con un resumen fijo a la derecha; la grilla marca los horarios ocupados y el "último lugar".
- **Estado en la URL.** Filtros del catálogo, reservas a medio completar y listados del admin se guardan en la dirección: si el cliente debe iniciar sesión, vuelve al mismo punto, y un link a una búsqueda se puede compartir.
- **Celular primero en lo que usa el cliente.** Sin desplazamiento horizontal, menú desplegable y tablas que se convierten en tarjetas.

Las capturas del sistema final están en el manual de usuario (sección 7).

6. Implementación del sistema

6.1 Estructura del proyecto

El repositorio contiene los dos proyectos (backend y frontend), una copia de los scripts de base de datos, la documentación técnica y la configuración de integración continua:

```
km0-automotores/
├── backend/                # API REST en Spring Boot 3.5 (Java 17)
│   ├── src/main/java/.../   # config, security, controller, service,
│   │                        # service/reglas, repository, model, dto...
│   ├── src/main/resources/db/migration/   # Flyway V1–V4 y fotos
│   └── src/test/            # unitarias e integración (Testcontainers)
├── frontend/               # SPA en React 19 + TypeScript + Vite
│   └── src/  api/ auth/ components/ hooks/ pages/ styles/ utils/
├── database/               # copia de los scripts SQL
├── docs/                   # arquitectura, ADR, diagramas, pruebas, uso de IA
├── scripts/fotos/          # descarga de fotos con licencia libre
├── .github/workflows/ci.yml # integración continua
└── docker-compose.yml      # PostgreSQL para desarrollo
```

En el backend cada módulo atraviesa las mismas capas: un controlador por recurso, un servicio con la lógica y uno o más repositorios. Los objetos que entran y salen de la API son `record` de Java en el paquete `dto`, separados de las entidades JPA, para que un cambio en la base no altere el contrato con el frontend. En el frontend, todas las llamadas al servidor pasan por `api/api.ts`, y los tipos de `api/tipos.ts` son el espejo de esos DTO.

6.2 Módulos principales

La Tabla 16 resume los ocho módulos con su responsabilidad, su interfaz pública y sus dependencias. A continuación se comentan los fragmentos de código que ilustran las decisiones más importantes.

Tabla 16. *Módulos del sistema*

Módulo	Responsabilidad	Interfaz pública (endpoints y pantallas)	Depende de
1. Autenticación y usuarios	Registro, inicio de sesión, token, roles, alta y baja de empleados	/api/auth/registro, /login, /yo; /api/admin/usuarios; pantallas Ingresar y Registro	—
2. Catálogo	Marcas, modelos, versiones, fotos, colores; comparador; visor 3D	/api/marcas, /api/modelos, /api/versiones; Catálogo, Ficha, Comparar	—
3. Sucursales y unidades	Sucursales con horario; unidades de demostración	/api/sucursales; /api/admin/sucursales, /unidades	2
4. Test drive	Disponibilidad, reserva, cancelación	/api/test-drives/disponibilidad, POST /api/test-drives, /mios, /{id}/cancelar; Reservar, Mis turnos	1, 2, 3

Módulo	Responsabilidad	Interfaz pública (endpoints y pantallas)	Depende de
5. Panel del vendedor	Agenda semanal, alertas, cambio de estado	/api/test-drives/agenda, /alertas, /{id}/estado; Mi panel	4
6. Métricas y recordatorios	Tablero, listado de reservas, aviso del día anterior	/api/admin/metricas; Admin → Métricas y Reservas; tarea programada	4, 7
7. Escuela de manejo	Clases con instructor elegido o asignado	/api/clases/…; Escuela de manejo; panel del instructor	1, 3
8. Solicitudes de compra	Presupuesto con simulación de cuotas y embudo de venta	/api/solicitudes-compra/…; Quiero comprarlo, Presupuesto, Admin → Ventas	1, 2, 3

Disponibilidad de horarios

Para cada bloque de una hora del día, el servicio calcula cuántas reservas más admite. Un horario solo se ofrece si hay a la vez una unidad del modelo y un vendedor libres, por eso los lugares son el mínimo entre ambos. Los bloques que empiezan en menos de una hora no se ofrecen (anticipación mínima).

```
for (LocalTime hora :
        CalculadoraHorarios.horariosDelDia(horario, fecha, duracion)) {
    List<OcupacionTestDrive> enEseHorario =
            ocupacion.getOrDefault(hora, List.of());
    int lugares = Math.min(
            unidadesLibres(unidades, enEseHorario).size(),
            vendedoresLibres(vendedores, enEseHorario).size());
    boolean alAlcance = CalculadoraHorarios.estaAlAlcance(
            fecha, hora, ahora, anticipacion);
    horarios.add(new HorarioDto(hora, alAlcance && lugares > 0,
            alAlcance ? lugares : 0));
}
```

`CalculadoraHorarios` es una clase pura: genera los bloques según el horario de la sucursal (el sábado hasta `cierre_sabado`, el domingo ninguno) y recibe la hora actual como parámetro, obtenida de un `Clock` en la zona de Córdoba. Eso permite probarla con fechas fijas, sin depender del día en que se ejecutan las pruebas.

Garantía contra turnos superpuestos

La reserva elige la primera unidad libre y el vendedor libre con menos turnos en el día, y guarda con `saveAndFlush` para que el INSERT se ejecute en ese momento. La unicidad real la garantiza la base con el índice parcial, que ignora los turnos cancelados:

```
CREATE UNIQUE INDEX uq_turno_vehiculo_horario
    ON turno_test_drive (vehiculo_demo_id, fecha, hora_inicio)
    WHERE estado <> 'CANCELADO';
```

Si otro pedido ganó el horario por milisegundos, PostgreSQL rechaza el INSERT y el manejador global de excepciones (`GlobalExceptionHandler`) traduce la violación de integridad a un 409

con el mensaje "Ese horario se acaba de ocupar. Elegí otro, por favor." El frontend lo muestra y vuelve a pedir la disponibilidad.

Simulación de la cuota

El frontend replica la fórmula del sistema francés para mostrar la cuota mientras el cliente mueve el anticipo. El valor que queda en el presupuesto lo calcula el backend con `BigDecimal`; las pruebas de ambos lados usan el mismo caso (un millón de pesos al 12 % anual en 12 cuotas da 88.848,79) para asegurar que coincidan.

```
export function cuotaMensual(
  monto: number, cuotas: number, tnaPorcentaje: number,
): number {
  if (cuotas <= 0)
    throw new Error('La cantidad de cuotas tiene que ser positiva')
  const i = tnaPorcentaje / 1200
  const cuota = i === 0 ? monto / cuotas : (monto * i) / (1 - (1 + i) ** -cuotas)
  return Math.round(cuota * 100) / 100
}
```

6.3 Integración con sistemas externos

Tabla 17. *Servicios externos*

Servicio y plan	Propósito	Autenticación	Seguridad y manejo de fallas
Wikimedia Commons (API pública)	Origen de las fotos de los modelos	Ninguna (contenido público)	Un script descarga solo imágenes con licencia libre (sin cláusulas NC ni ND) y guarda autor y licencia; las fotos se sirven desde el propio sitio. Si una no carga, se muestra una silueta.
Servidor SMTP (opcional, por ejemplo Gmail)	Recordatorios y avisos por email	Usuario y contraseña de aplicación en variables de entorno	Credenciales fuera del código. Si el envío falla o está deshabilitado, el mensaje se registra en el log y la operación del usuario no se interrumpe.
Render (plan gratuito)	Backend en contenedor Docker y base PostgreSQL	Cuenta del equipo; variables de entorno en el panel	HTTPS automático; secretos (JWT_SECRET, DB_PASSWORD) solo en el panel. El servicio gratuito se suspende sin uso y tarda cerca de un minuto en despertar.
Vercel (plan Hobby)	Frontend estático	Cuenta vinculada a GitHub	HTTPS y CDN incluidos. `vercel.json` redirige todas las rutas a la SPA para que los links directos funcionen.

7. Manual de usuario

7.1 Introducción al manual

Km0 Automotores es el sitio web de una concesionaria de autos 0 km de Córdoba. Desde el sitio usted puede ver y comparar modelos, girarlos en 3D, reservar un test drive o una clase de manejo y pedir un presupuesto. Los empleados lo usan para ver su agenda y administrar el catálogo. La Figura 7 muestra la pantalla principal.

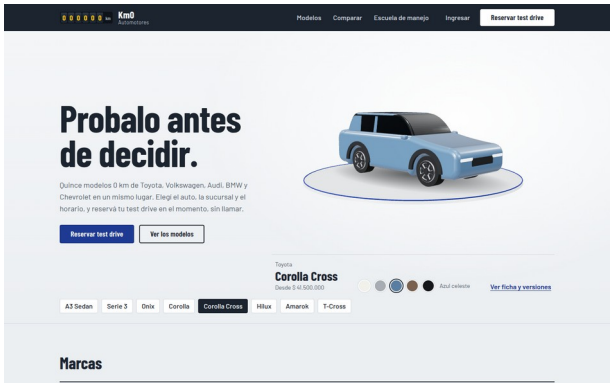


Figura 7. *Pantalla principal del sitio*
Fuente: Elaboración propia.

7.2 Acceso al sistema

Ingrese desde cualquier navegador actualizado (Chrome, Edge, Firefox o Safari), en computadora o celular, a la dirección https://km0-automotores-prueba.vercel.app. No necesita instalar nada.

Para crear una cuenta, haga clic en "Ingresar" y luego en "Creala en un minuto". Complete nombre, apellido, email, teléfono y una contraseña de al menos 8 caracteres, y presione "Crear cuenta". Puede ver el catálogo sin cuenta; la necesita para reservar o pedir un presupuesto.

Para iniciar sesión, haga clic en "Ingresar", escriba su email y contraseña y presione "Ingresar". El sistema lo lleva a la pantalla que corresponde a su perfil. Para probar el sistema puede usar las cuentas de prueba que figuran a la derecha de esa pantalla; la contraseña de todas es `Demo1234`.

Tabla 18. *Perfiles de usuario*

Perfil	Qué puede hacer	Cuenta de prueba
Cliente	Reservar test drives y clases, ver y cancelar sus turnos, pedir presupuestos	cliente@example.com
Vendedor	Ver su agenda, confirmar turnos, registrar asistencia y seguir sus solicitudes de compra	martin.ferreyra@km0.com.ar
Instructor	Ver su agenda de clases y registrar asistencia	valeria.ponce@km0.com.ar
Administrador	Todo lo anterior para cualquier empleado, más métricas, reservas, ventas y mantenimiento del catálogo	admin@km0.com.ar

7.3 Descripción de funcionalidades

Ver el catálogo y la ficha de un modelo

1. Haga clic en "Modelos" en el menú superior.
2. Use los filtros de la izquierda (marca, carrocería, combustible, precio máximo) para acotar la lista (Figura 8, izquierda).
3. Haga clic en un modelo para abrir su ficha. Allí verá el precio, las versiones con su ficha técnica, las fotos y el visor 3D.
4. En la pestaña "Girar en 3D", arrastre el auto con el mouse o el dedo para girarlo, y elija un color debajo para pintarlo (Figura 8, derecha).

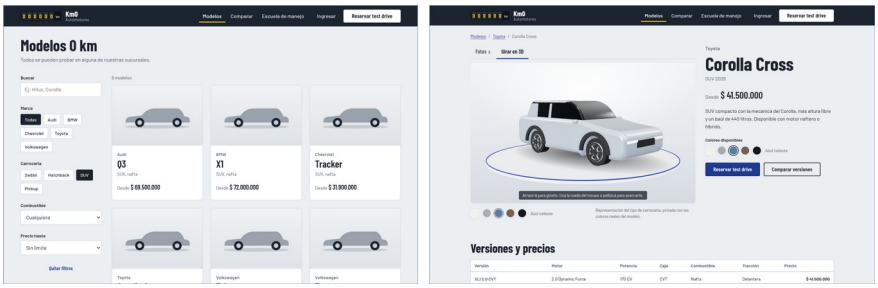


Figura 8. *Catálogo con filtros (izquierda) y ficha de un modelo con el visor 3D (derecha)*

Fuente: Elaboración propia.

Comparar versiones

Haga clic en "Comparar", elija hasta tres versiones en los selectores y revise las especificaciones en paralelo (Figura 10, izquierda). Las mejores cifras de cada fila aparecen resaltadas.

Reservar un test drive

1. Haga clic en "Reservar test drive" (arriba a la derecha) o en el botón del mismo nombre dentro de la ficha del modelo.
2. Elija el modelo, la sucursal y el día. Los días en que la sucursal no atiende aparecen deshabilitados.
3. Elija un horario libre. Los ocupados aparecen tachados y los que tienen un solo lugar dicen "Último lugar".
4. Si quiere, escriba un comentario para el vendedor y presione "Confirmar reserva". Si no inició sesión, el sistema le pide ingresar y luego vuelve a su reserva.
5. Verá la confirmación con el vendedor asignado (Figura 9, izquierda). Un día antes recibirá un recordatorio.

Ver y cancelar sus turnos

Haga clic en "Mis turnos" (Figura 9, derecha). Verá sus test drives, clases y solicitudes de compra, con su estado. Para cancelar un turno que todavía no empezó, presione "Cancelar turno" y confirme. El horario queda libre para otra persona.

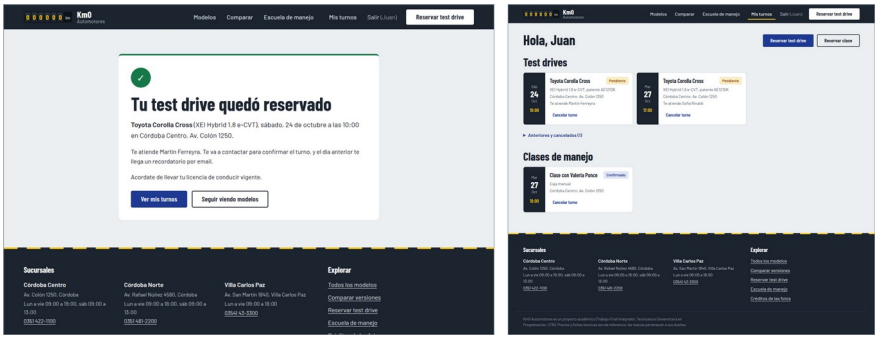


Figura 9. *Confirmación de la reserva (izquierda) y "Mis turnos" (derecha)*

Fuente: Elaboración propia.

Reservar una clase de manejo

Haga clic en "Escuela de manejo". Elija la sucursal, el tipo de caja (manual o automática) y un instructor, o deje "El que esté libre". Luego elija día y horario y presione "Reservar clase" (Figura 10, derecha).

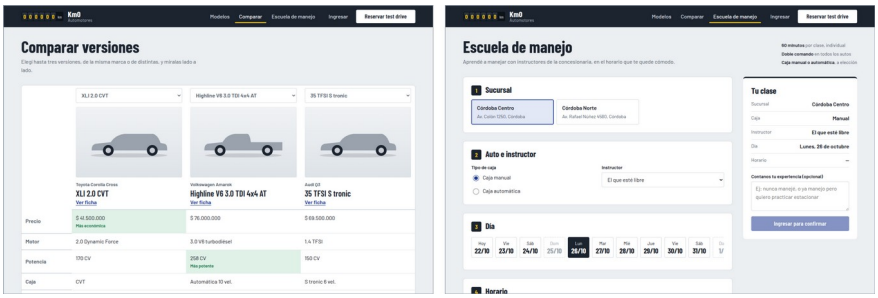


Figura 10. *Comparador de versiones (izquierda) y reserva de clase de manejo (derecha)*

Fuente: Elaboración propia.

Pedir un presupuesto de compra

1. En la ficha del modelo, presione "Quiero comprarlo".
2. Elija la versión, el color y la forma de pago. Si elige "Financiado", mueva el anticipo y elija la cantidad de cuotas; la cuota estimada se actualiza a la derecha (Figura 11, izquierda).
3. Marque "Entrego mi auto usado" si corresponde y describa el auto. Elija la sucursal y presione "Enviar solicitud".
4. Verá su presupuesto con el vendedor asignado (Figura 11, derecha). Presione "Imprimir o guardar PDF" para conservarlo; es orientativo y no se paga nada en el sitio. Si cambia de idea, presione "Ya no me interesa".

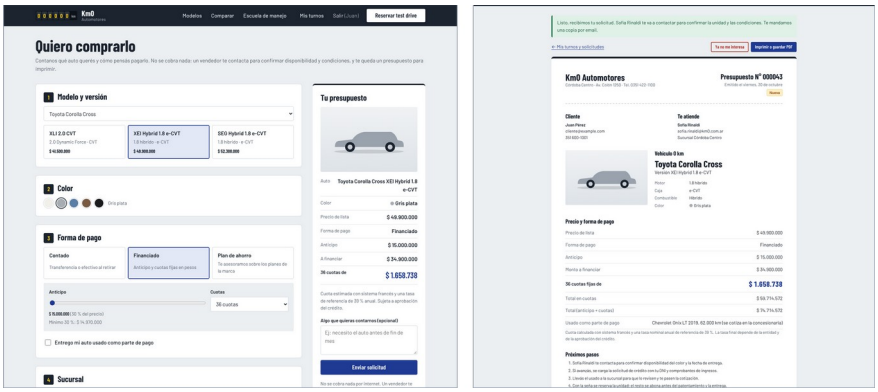


Figura 11. *Formulario "Quiero comprarlo" con la simulación de cuotas (izquierda) y presupuesto listo para imprimir (derecha)*

Fuente: Elaboración propia.

Panel del vendedor y del instructor

Al ingresar, el vendedor ve "Mi agenda" (Figura 12). Arriba, en amarillo, están los turnos de los próximos días que todavía no confirmó. Llame al cliente y presione "Confirmar". Cuando el turno haya empezado, presione "Vino" o "No vino". Use las flechas de "Semana" para moverse entre semanas. Más abajo, en "Solicitudes de compra", avance cada interesado por las etapas Contactada, Reservada y Vendida, o márquelo como Descartada. El instructor tiene el mismo panel para sus clases.

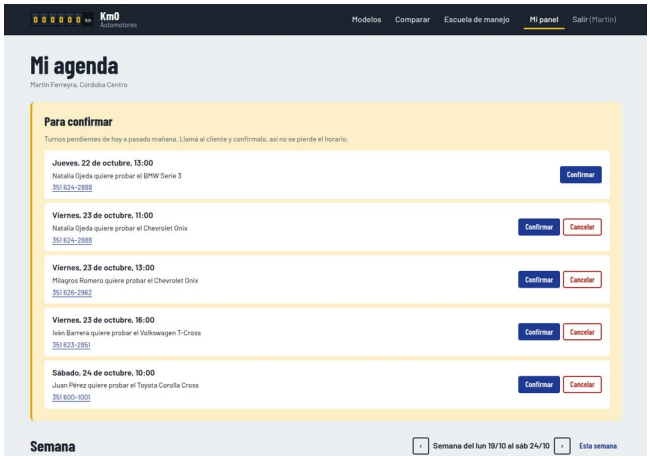


Figura 12. *Agenda del vendedor con alertas*

Fuente: Elaboración propia.

Panel del administrador

El administrador accede a "Mi panel", con un menú de secciones:

- **Métricas** (Figura 13, izquierda): elija el período para ver turnos por estado, modelos y horarios más pedidos, asistencia y turnos por sucursal.
- **Reservas** (Figura 13, derecha): todos los test drives y clases del período. Filtre por fechas, tipo, sucursal o estado, busque por cliente o teléfono y presione "Descargar Excel (CSV)" para exportar.

- **Ventas** (Figura 14, izquierda): todas las solicitudes de compra, con la conversión y el monto vendido.
- **Catálogo, Unidades demo, Sucursales y Usuarios**: para dar de alta, modificar o dar de baja cada elemento (Figura 14, derecha). Los cambios se ven de inmediato en el sitio.

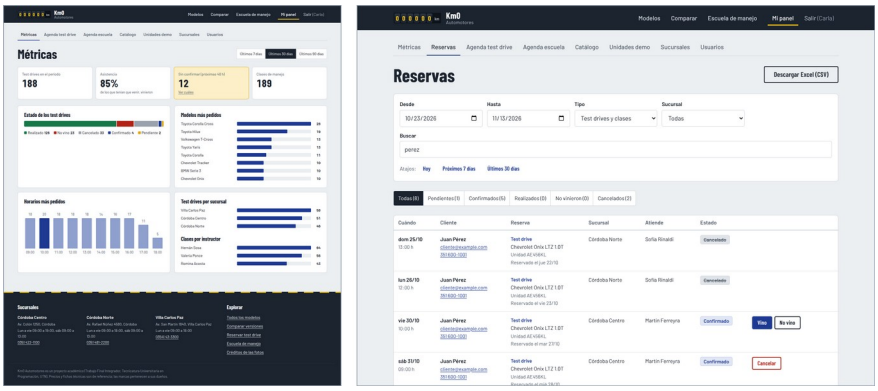


Figura 13. *Tablero de métricas (izquierda) y listado de reservas con filtros (derecha)*

Fuente: Elaboración propia.

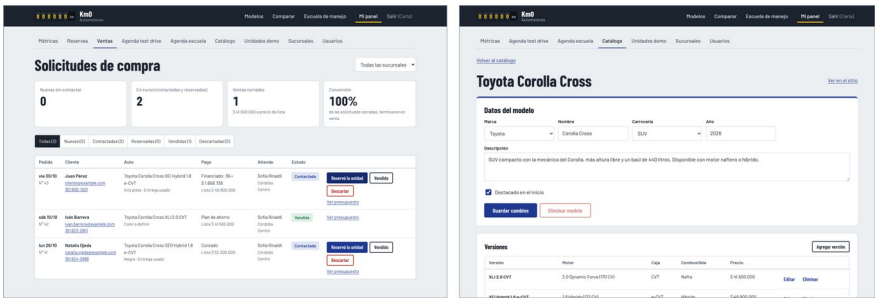


Figura 14. *Seguimiento de solicitudes de compra (izquierda) y edición de un modelo del catálogo (derecha)*

Fuente: Elaboración propia.

Mensajes de error frecuentes

Tabla 19. *Mensajes frecuentes y cómo resolverlos*

Mensaje	Qué hacer
"Ese horario se acaba de ocupar. Elegí otro, por favor."	Otra persona reservó ese horario un instante antes. Elija otro de la grilla, que ya se actualizó.
"Ya tenés otro turno reservado en ese horario."	Usted tiene otro test drive o clase en ese horario. Elija otro horario o cancele el turno anterior.
El sitio lo lleva a la pantalla "Ingresar" mientras usaba el sistema	Su sesión venció (dura 24 horas). Vuelva a ingresar con su email y contraseña.
"Email o contraseña incorrectos."	Revise lo escrito, incluidas mayúsculas. Si no tiene cuenta, créela desde "Creala en un minuto".
"El anticipo mínimo es el 30 % del precio…"	Aumente el anticipo con el control deslizante o elija otra forma de pago.
"No pudimos conectarnos con el servidor. Revisá tu conexión e intentá de nuevo."	Revise su conexión y espere un minuto: el servidor puede estar iniciándose. Luego presione "Reintentar".

7.4 Preguntas frecuentes

- **¿Tengo que pagar algo para reservar un test drive?** No. La reserva es gratuita y no se pide ningún medio de pago.
- **¿Puedo elegir el vendedor que me atiende?** En el test drive, no: el sistema asigna al vendedor libre con menos turnos ese día. En la escuela de manejo sí puede elegir instructor.
- **¿Qué pasa si no puedo ir?** Cancele el turno desde "Mis turnos" antes de que empiece, así el horario queda libre para otra persona.
- **¿La cuota del presupuesto es definitiva?** No. Se calcula con una tasa de referencia; la tasa final depende de la entidad financiera y de la aprobación del crédito.

8. Manual de implementación

Este manual permite instalar el sistema en una computadora para desarrollo y publicarlo en la nube. Supone conocimientos básicos de línea de comandos y de Git, pero no haber participado del proyecto.

8.1 Requisitos del entorno

Tabla 20. *Requisitos del entorno*

Categoría	Detalle
Sistema operativo	Windows 10/11, macOS 13 o Ubuntu 22.04 LTS o superiores para desarrollo. En producción, el contenedor usa Linux (imagen eclipse-temurin:17-jre).
Hardware mínimo	Desarrollo: 8 GB de RAM y 5 GB libres en disco (Docker incluido). Producción: 512 MB de RAM y 0,1 CPU para el backend (plan gratuito de Render); la base ocupa menos de 50 MB.
Software base	Java 17 (JDK Temurin), Maven 3.9 o superior, Node.js 20 o superior (CI usa 22) con npm, Git, Docker Desktop.
Base de datos	PostgreSQL 16 con una base vacía; Flyway crea las tablas al arrancar.
Servidor web / proxy	No requiere uno propio: el backend incluye Tomcat embebido y Vercel y Render proveen HTTPS.
Puertos	5432 (PostgreSQL), 8080 (backend; en Render lo define la variable PORT), 5173 (frontend en desarrollo).
Conectividad	Internet para descargar dependencias y, opcionalmente, para enviar emails por SMTP. El sistema funciona sin acceso a servicios externos.

8.2 Variables de entorno y configuración

Todas las variables tienen un valor por defecto pensado para desarrollo local, por lo que en la computadora del desarrollador no hace falta definir ninguna. En producción se cargan en el panel de cada plataforma. El archivo `backend/.env.example` las lista con ejemplos.

Tabla 21. *Variables de entorno*

Variable	Descripción	Ejemplo (nunca un secreto real)	¿Obligatoria? (por defecto)
DB_URL	Conexión JDBC a PostgreSQL	jdbc:postgresql://host:5432/km0_automotores	Sí en producción (localhost)
DB_USER, DB_PASSWORD	Usuario y contraseña de la base	km0 / ********	Sí en producción (postgres)
JWT_SECRET	Clave para firmar los tokens; mínimo 32 caracteres aleatorios	(generar con un gestor de contraseñas)	Sí en producción
CORS_ORIGINS	Dominios del frontend autorizados, separados por coma	https://km0-automotores-prueba.vercel.app	Sí en producción (localhost:5173)
PORT	Puerto HTTP del backend	8080	No (Render la define)
MAIL_ENABLED	Habilita el envío real de emails	false	No (false)
SPRING_MAIL_HOST, _PORT, _USERNAME, _PASSWORD	Datos del servidor SMTP	smtp.gmail.com / 587 / cuenta / clave de aplicación	Solo si se habilita el email
VITE_API_URL (frontend)	URL pública del backend, sin barra final	https://km0-backend-prueba.onrender.com	Sí en producción (localhost:8080)

Los parámetros de negocio (duración de los turnos, anticipación mínima, días máximos de adelanto, tasa de referencia, plazos y anticipo mínimo) están en `backend/src/main/resources/application.yml`, en la sección `app`.

8.3 Procedimiento de instalación

Instalación local (desarrollo o demostración)

Paso 1. Clonar el repositorio.

```
git clone https://github.com/AgusS05/km0-automotores.git
cd km0-automotores
```

Verificación: la carpeta contiene `backend`, `frontend`, `docs` y `docker-compose.yml`.

Paso 2. Levantar la base de datos. Abra Docker Desktop y espere a que indique que está en ejecución. Luego:

```
docker compose up -d db
docker compose ps
```

Verificación: el contenedor `km0-db` figura con estado `running (healthy)`. Si aparece el error "failed to connect to the docker API", Docker Desktop no está abierto.

Paso 3. Iniciar el backend. Las dependencias se descargan solas la primera vez. Flyway crea las tablas y carga los datos de ejemplo.

```
cd backend
mvn spring-boot:run
```

Verificación: el log muestra `Successfully applied 6 migrations` (solo la primera vez: V1 a V5 y las fotos) y `Tomcat started on port 8080`. En el navegador, `http://localhost:8080/api/marcas` devuelve un JSON con 6 marcas y `http://localhost:8080/swagger-ui.html` muestra la documentación de la API.

Paso 4. Iniciar el frontend. En otra terminal:

```
cd frontend
npm install
npm run dev
```

Verificación: la consola muestra `Local: http://localhost:5173/`. Al abrir esa dirección se ve el inicio con el auto 3D, y es posible ingresar con `cliente@example.com` / `Demo1234`.

Paso 5 (opcional). Descargar las fotos reales. Requiere internet; descarga tres fotos por modelo con licencia libre y genera los créditos. Al reiniciar el backend, Flyway las registra.

```
node scripts/fotos/descargar-fotos.mjs
```

Despliegue en producción

1. **Base de datos.** En Neon, cree un proyecto con PostgreSQL 16 en la región AWS US East 2 (Ohio). En Connect, desactive Connection pooling (Flyway necesita una conexión directa) y copie la cadena de conexión. Arme `DB_URL` como `jdbc:postgresql://HOST/neondb?sslmode=require`, sin usuario ni contraseña, que van en `DB_USER` y `DB_PASSWORD`. No se usa la base gratuita de Render porque se elimina a los 30 días.
2. **Backend.** En Render, cree un Web Service desde el repositorio de GitHub con Root Directory `backend` y entorno Docker (usa el `Dockerfile` incluido). Región Ohio, instancia Free y Health Check Path `/api/marcas`. Cargue `DB_URL`, `DB_USER`, `DB_PASSWORD`, `JWT_SECRET` (botón Generate) y `CORS_ORIGINS` con la URL prevista del frontend. Verificación: al terminar el deploy, `https://<servicio>.onrender.com/api/marcas` devuelve las 6 marcas.
3. **Frontend.** En Vercel, importe el repositorio con Root Directory `frontend` (Vercel detecta Vite: build `npm run build`, salida `dist`). Defina `VITE_API_URL` con la URL del backend. Verificación: la URL de Vercel muestra el catálogo con los 21 modelos.
4. **CORS.** Vuelva a Render y cambie `CORS_ORIGINS` por la URL de Vercel (puede dejar también la de localhost, separadas por coma). Render reinicia el servicio. Verificación: el inicio de sesión desde Vercel funciona y la consola del navegador no muestra errores de CORS.

5. **Prueba final.** Recorra los casos E2E de la sección 9.2 con las cuentas de prueba.

> Despliegue actual (06/11/2026). Frontend: https://km0-automotores-prueba.vercel.app · Backend: https://km0-backend-prueba.onrender.com · Base de datos: Neon (AWS US East 2).

8.4 Actualización del sistema

Render y Vercel vuelven a desplegar automáticamente cada vez que se integra un cambio en `main`. Los cambios de esquema se agregan como una nueva migración (`V5__descripcion.sql`, nunca editando una ya aplicada), y Flyway la aplica una sola vez al arrancar la nueva versión, sin borrar datos. Antes de integrar, la integración continua debe estar en verde.

Rollback. Si una versión falla, en Render abra el servicio → Events y elija "Rollback" sobre el deploy anterior; en Vercel, Deployments → deploy anterior → "Promote to Production". Si la versión defectuosa incluía una migración, el rollback del código no la revierte: escriba una migración correctiva (`V6__...`) o restaure el respaldo previo (8.5). Por eso se recomienda hacer un respaldo antes de desplegar cambios de esquema.

8.5 Respaldo y recuperación

Se recomienda un respaldo diario automático (Render y Neon los ofrecen según el plan) y uno manual antes de cada cambio de esquema. El respaldo manual se hace con las herramientas de PostgreSQL:

```
DB="postgresql://USUARIO:CLAVE@HOST:5432/BASE"
pg_dump -Fc -d "$DB" -f km0-AAAAMMDD.dump          # respaldo
pg_restore --clean --no-owner -d "$DB" km0-AAAAMMDD.dump   # restauración
```

Verificación de la restauración: reiniciar el backend y comprobar que arranca sin errores de Flyway y que `/api/modelos` devuelve los datos esperados. El sistema no genera archivos propios: las fotos son parte del repositorio.

8.6 Monitoreo y logs

El backend escribe sus logs en la salida estándar, visibles en la pestaña Logs del servicio en Render (o en la terminal en desarrollo). En producción se recomienda el nivel `INFO`; para diagnosticar un problema puntual puede definirse la variable `LOGGING_LEVEL_COM_KM0AUTOMOTORES` con el valor `DEBUG` y volver a `INFO` al terminar. Cada error de negocio queda registrado con la URL del pedido, y los recordatorios enviados (o simulados, si el email está deshabilitado) también se registran. La Tabla 22 describe los mensajes más comunes.

Tabla 22. *Errores comunes en el log y su solución*

Mensaje	Causa y solución
Connection to localhost:5432 refused	La base no está levantada o DB_URL apunta a otro lugar. Iniciar Docker y `docker compose up -d db`, o revisar DB_URL.
FlywayValidateException: Migration checksum mismatch	Se editó una migración ya aplicada. Revertir el cambio y escribirlo en una migración nueva.
Schema-validation: missing column / table	Una entidad no coincide con la base. Falta una migración o tiene un error.
Violación de integridad en /api/test-drives	Dos reservas simultáneas del mismo horario. Es el comportamiento esperado (409); no requiere acción.
El navegador informa "blocked by CORS policy"	CORS_ORIGINS no incluye la URL exacta del frontend (sin barra final).
Todas las llamadas devuelven 401 tras un deploy	Cambió JWT_SECRET y los tokens anteriores dejaron de ser válidos. Los usuarios deben volver a ingresar.

9. Pruebas y validación

9.1 Estrategia de pruebas

La estrategia sigue la pirámide de pruebas: la mayor cantidad de pruebas está en las reglas de negocio puras, que son rápidas y deterministas; un conjunto menor de pruebas de integración recorre la API completa contra un PostgreSQL real; y los recorridos de punta a punta se hacen manualmente por rol. Todas las pruebas automatizadas se ejecutan en GitHub Actions en cada push. La Tabla 23 indica qué tipos se incluyeron.

Tabla 23. *Tipos de prueba incluidos*

Tipo de prueba	¿Incluida?	Herramienta	Qué cubre
Unitarias (backend)	Sí	JUnit 5, Mockito	Horarios, estados, financiación; servicio de reservas con repositorios simulados
Integración (backend)	Sí	Spring Boot Test, MockMvc, Testcontainers	La API completa contra PostgreSQL 16: seguridad, controladores, servicios, JPA, Flyway e índices
Unitarias y de componentes (frontend)	Sí	Vitest, Testing Library	Formatos, cliente HTTP, simulación de cuotas, grilla de horarios, botones de estado, rutas protegidas
Sistema (E2E)	Sí, manual	Navegador	Recorridos completos por rol (tabla 9.2)
Usabilidad	[COMPLETAR: Sí/No]	—	Ver 9.4
Rendimiento	No	—	Fuera de alcance; se mitigó con diseño de consultas e índices

Tipo de prueba	¿Incluida?	Herramienta	Qué cubre
Seguridad	Parcial	Pruebas de integración	401 sin token, 403 por rol y sobre datos ajenos, validación de entradas, contraseñas con BCrypt
Análisis estático	Sí	oxlint, javac con doclint	Errores de React, accesibilidad, Javadoc

Las pruebas fijan la fecha mediante un `Clock` inyectado, y las de integración se omiten (no fallan) si la máquina no tiene Docker.

9.2 Casos de prueba

Se presentan los casos más representativos de cada tipo; el listado completo (24 unitarias, 18 de integración y 13 grupos del frontend) está en `docs/pruebas.md`.

Tabla 24. *Casos de prueba unitarios representativos*

ID	Descripción y precondición	Resultado esperado	Obtenido
UT-01	Día hábil en una sucursal de 9 a 19 h	Bloques de una hora de 9 a 18 h	Pasa
UT-06	Reserva con menos de una hora de anticipación	Rechazada con su motivo	Pasa
UT-13	Reserva con dos unidades y dos vendedores libres	Primera unidad y vendedor con menos turnos	Pasa
UT-15	Todas las unidades ocupadas en el horario	ConflictoException (409)	Pasa
UT-19	Cuota de 1.000.000 al 12 % anual en 12 cuotas	88.848,79	Pasa

Tabla 25. *Casos de prueba de integración representativos (API contra PostgreSQL)*

ID	Pasos	Resultado esperado	Obtenido
IT-01	GET /api/modelos sin token	200 con los 15 modelos cargados por Flyway	[COMPLETAR]
IT-08	POST /api/test-drives sin encabezado Authorization	401	[COMPLETAR]
IT-11	A reserva; B pide el mismo horario; B intenta cancelar el turno de A; A cancela; B reserva	201, 409, 403, 200 (CANCELADO), 201	[COMPLETAR]
IT-14	Dos INSERT del mismo vehículo y horario; cancelar el primero; repetir	El segundo falla; tras cancelar se acepta	[COMPLETAR]
IT-17	Solicitud financiada al 40 %; repetirla; otro cliente intenta verla; el vendedor la avanza	201, 409, 403, 200 y 422 en saltos inválidos	[COMPLETAR]

> Los resultados de integración se completan con la salida de `mvn verify` (o del job "Backend" de GitHub Actions), que es la primera ejecución completa con Maven y Docker.

Tabla 26. *Casos de prueba de sistema (manuales)*

ID	Rol	Pasos	Resultado esperado	Obtenido
E2E-01	Visitante	Inicio → elegir un modelo → "Girar en 3D" → cambiar color	El auto gira y cambia de color	[COMPLETAR]
E2E-02	Visitante	Reservar sin sesión → registrarse → volver → confirmar	Vuelve al paso de la reserva y queda confirmada con vendedor asignado	[COMPLETAR]
E2E-03	Cliente	Mis turnos → cancelar un turno futuro	Pasa a Cancelado y el horario vuelve a estar libre	[COMPLETAR]
E2E-04	Vendedor	Panel → "Para confirmar" → Confirmar	El turno sale de las alertas y aparece confirmado	[COMPLETAR]
E2E-05	Cliente	Quiero comprarlo → financiado → enviar → imprimir	Presupuesto con la cuota de la pantalla; PDF sin menús	[COMPLETAR]
E2E-07	Admin	Reservas → filtrar pendientes → Descargar Excel	El archivo abre en Excel con las mismas filas	[COMPLETAR]

9.3 Métricas de calidad

Tabla 27. *Métricas de calidad*

Métrica	Valor
Pruebas automatizadas	Backend: 24 unitarias y 18 de integración. Frontend: 47 pruebas unitarias y de componentes, todas aprobadas.
Cobertura de líneas, frontend (Vitest)	51 % del código medido. 94 % a 100 % en los archivos con lógica (formato de fechas y precios, cliente HTTP, grilla de horarios, botones de estado, rutas protegidas). Las pantallas sin lógica propia no tienen pruebas.
Cobertura de líneas, backend (JaCoCo)	[COMPLETAR con el valor de target/site/jacoco/index.html o del artefacto "cobertura-backend" de GitHub Actions]
Análisis estático, frontend (oxlint)	Primera pasada: 8 errores de accesibilidad (celdas de encabezado sin texto para lectores de pantalla), corregidos. Quedan 7 advertencias de React documentadas como deuda técnica: 5 por actualizar estado dentro de un efecto y 2 por dependencias intencionales.
Análisis estático, backend (doclint)	2 errores de HTML en comentarios Javadoc, corregidos; 0 errores.
Defectos encontrados y resueltos	11 en total (Tabla siguiente): 4 en el incremento 2, 2 en el 3 y 5 en el 4.
Rendimiento	No se realizaron pruebas de carga. [COMPLETAR opcional: tiempo de respuesta medido en producción con las herramientas del navegador]

Tabla 28. *Defectos encontrados por incremento*

#	Incr.	Defecto	Cómo se detectó	Corrección
1	2	Un turno cancelado bloqueaba el horario	Revisión del esquema al	Índices únicos parciales (ADR-

#	Incr.	Defecto	Cómo se detectó	Corrección
		para siempre	implementar la cancelación	04)
2	2	Dos clientes podían pasar la validación del mismo horario	Análisis de concurrencia	saveAndFlush e índice único; respuesta 409
3	2	La validación de fechas usaba la hora UTC del servidor y rechazaba turnos válidos de la noche	Prueba con fechas límite	Validación en el servicio con Clock en hora argentina
4	2	Tras cerrar sesión en una página protegida, el siguiente ingreso volvía a esa página	Prueba manual	Cerrar sesión lleva al inicio
5	3	Un cliente de los datos de ejemplo tenía un test drive y una clase a la misma hora	Consulta de verificación	El script de datos verifica ambas tablas
6	3	En el celular el auto 3D quedaba cortado	Captura en tamaño de celular	La cámara se aleja según la proporción de la pantalla
7	4	Celdas de encabezado sin texto accesible	oxlint	Texto oculto visualmente para lectores de pantalla
8	4	El formulario de registro desbordaba su tarjeta	Uso en Windows por el equipo	Corrección de los estilos de los campos
9	4	El menú de administración desaparecía al entrar en las agendas	Uso por el equipo	Rutas anidadas dentro del panel de administración
10	4	En Usuarios no se podía volver de "Clientes" a "Empleados"	Uso por el equipo	El tipo elegido se guarda en la URL
11	4	El presupuesto impreso salía pegado a los bordes de la hoja	Impresión a PDF	Márgenes propios en los estilos de impresión

9.4 Validación con usuarios

[COMPLETAR: si hicieron pruebas con usuarios (familiares, compañeros, alguien que haya comprado un auto), describir quiénes participaron y su perfil, qué tareas se les pidió (por ejemplo: "reservá un test drive del Corolla Cross para el sábado"), qué problemas encontraron y qué se cambió. Si no se hicieron, decirlo y explicar por qué; la cátedra valora la honestidad.]

Durante el desarrollo, los propios integrantes usaron el sistema en sus computadoras con Windows como lo haría un usuario, y de ese uso surgieron los defectos 8 a 11 de la Tabla 28.

10. Conclusiones

10.1 Resultados obtenidos

Tabla 29. *Cumplimiento de los objetivos específicos*

Objetivo específico	Estado	Justificación
1. Catálogo de 15 modelos con filtros, fichas, fotos, visor 3D y comparador	Cumplido	Implementado con 6 marcas, 21 modelos y 50 versiones. El visor es estilizado por carrocería (ADR-06), no una réplica de cada modelo.
2. Reserva con disponibilidad real y sin superposiciones	Cumplido	La garantía la da la base con índices parciales; verificado con pruebas de integración y a mano en PostgreSQL.
3. Agenda de vendedores e instructores con alertas y asistencia	Cumplido	Agenda semanal, alertas y cambio de estado con reglas validadas en el servidor.
4. Métricas, listado de reservas y mantenimiento del catálogo	Cumplido	Incluye exportación a Excel y, como extensión, el seguimiento de ventas.
5. Despliegue en la nube con integración continua y pruebas	[COMPLETAR: Cumplido / Parcial]	La integración continua y las pruebas están implementadas, y el sistema está publicado en Vercel, Render y Neon desde el 06/11/2026. [COMPLETAR: resultado del primer mvn verify]

Además de los objetivos planteados, se agregaron la escuela de manejo (prevista como opcional), las fotos reales con verificación de licencia y el módulo de solicitudes de compra.

10.2 Lecciones aprendidas

Técnicas. La lección más valiosa fue que una regla de negocio importante ("no puede haber dos turnos en el mismo horario") no se garantiza solo con código: la validación del servicio sirve para dar un buen mensaje, pero la garantía real tiene que estar en la base. También aprendimos a desconfiar de la hora: un servidor en otra zona horaria rompe validaciones que parecían correctas, y la solución (inyectar un reloj) además hizo posibles las pruebas con fechas fijas.

Metodológicas. Trabajar con incrementos que funcionan de punta a punta permitió mostrar avances reales al tutor cada pocos días y detectar temprano el problema del esquema aprobado. Documentar los cambios respecto de lo aprobado, en lugar de modificarlos en silencio, evitó discusiones en la revisión.

Uso de inteligencia artificial. El asistente aceleró la escritura de código y documentación, pero cometió errores que solo se detectaron probando (Anexo A): el trabajo pasa de escribir a revisar y entender lo generado, y eso exige conocer el dominio y el stack.

Qué haríamos distinto. Desplegaríamos desde el primer incremento: problemas como CORS o las variables de entorno aparecen recién fuera de la computadora de desarrollo. También usaríamos pull requests desde el primer día y escribiríamos las pruebas de integración junto con cada endpoint.

Trabajo en equipo. Dividir el trabajo por capa (frontend, backend, base de datos y calidad) permitió avanzar en paralelo, pero obligó a acordar temprano el contrato de la API; las reuniones semanales sirvieron para detectar los cambios que afectaban a otra capa. **[REVISAR: ajustar esta reflexión con la experiencia real del equipo]**

10.3 Limitaciones del sistema actual

- No hay cobro en línea ni conexión con entidades financieras: la cuota es una simulación con tasa de referencia.
- El token de sesión se guarda en `localStorage` (riesgo ante XSS) y vale hasta que vence (ADR-03).
- El visor 3D es estilizado; no reproduce el diseño exacto de cada modelo.
- En el plan gratuito de Render el backend se suspende sin uso y tarda cerca de un minuto en responder la primera vez.
- No se realizaron pruebas de rendimiento ni **[COMPLETAR: de usabilidad con usuarios externos]**.

10.4 Trabajo futuro

- Mover el token a una cookie `HttpOnly` con protección CSRF.
- Integrar un simulador de crédito real de una entidad financiera y una seña en línea para reservar la unidad.
- Pruebas de punta a punta automatizadas con Playwright en la integración continua.
- Modelos 3D reales para los modelos destacados, cargados en formato glTF, manteniendo el visor actual como alternativa.

11. Referencias bibliográficas

Ambler, S. W., & Sadalage, P. J. (2006). *Refactoring databases: Evolutionary database design*. Addison-Wesley.

Anderson, D. J. (2010). *Kanban: Successful evolutionary change for your technology business*. Blue Hole Press.

Anthropic. (2026). *Claude* [Modelo de lenguaje de gran escala]. https://claude.ai

Brown, S. (s. f.). *The C4 model for visualising software architecture*. Recuperado el 30 de octubre de 2026, de https://c4model.com

Cohn, M. (2009). *Succeeding with agile: Software development using Scrum*. Addison-Wesley.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

OWASP Foundation. (2021). *OWASP Top 10: 2021*. https://owasp.org/Top10/

PostgreSQL Global Development Group. (s. f.). *PostgreSQL 16 documentation: Partial indexes*. Recuperado el 30 de octubre de 2026, de https://www.postgresql.org/docs/16/indexes-partial.html

React. (s. f.). *React documentation*. Recuperado el 30 de octubre de 2026, de https://react.dev

Redgate. (s. f.). *Flyway documentation*. Recuperado el 30 de octubre de 2026, de https://documentation.red-gate.com/flyway

Spring. (s. f.). *Spring Boot reference documentation* (versión 3.5). Recuperado el 30 de octubre de 2026, de https://docs.spring.io/spring-boot/

Testcontainers. (s. f.). *Testcontainers for Java*. Recuperado el 30 de octubre de 2026, de https://java.testcontainers.org

three.js. (s. f.). *three.js documentation*. Recuperado el 30 de octubre de 2026, de https://threejs.org/docs/

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/

Anexo A. Uso de inteligencia artificial

La cátedra pide documentar cómo se incorporó la inteligencia artificial al desarrollo. Se utilizó Claude (Anthropic, 2026) como asistente de desarrollo, además de springdoc para documentar la API y oxlint para el análisis estático. El registro completo, con el detalle de qué revisó cada integrante, está en `docs/uso-de-ia.md`.

El equipo definió el problema, la propuesta de valor, el stack y los módulos, revisó cada entrega con el tutor y probó el sistema. A partir de esas definiciones, la IA generó el código, el esquema SQL con sus datos de ejemplo, las pruebas, la documentación técnica, los diagramas y el borrador de este informe.

Validación. Nada se dio por bueno sin comprobarlo: el esquema se ejecutó en PostgreSQL 16, el código se compiló y se probó, y los recorridos se verificaron en el navegador. Entre los errores detectados figuran un script de PL/pgSQL con un error de sintaxis y fechas validadas en hora UTC. No se compartieron datos reales con la IA. **[COMPLETAR: otras IA usadas]**